Learning to Select Actions for Complex Hand Tasks: A Comparative Study of Visuomotor Policies and Latent World Models

Graham Pellegrini

Supervisor: Prof. Alexiei Dingli

[PLACEHOLDER: Submission Date]

Submitted in partial fulfilment of the requirements for the degree of Master of Science in Artificial Intelligence.

L-Università ta' Malta
Faculty of Information & Communication Technology

Plagiarism Declaration

I, the undersigned, hereby declare that the work contained in this dissertation is my own original work and that I have not previously in its entirety or in part submitted it at any university for a degree.

I declare that this submission is my own work and that, to the best of my knowledge and belief, it contains no material previously published or written by another person, except where due acknowledgement is made in the text.

I declare that the use of artificial intelligence tools in the preparation of this dissertation was limited to writing assistance, code suggestions, and literature search support. All research design, experimental decisions, implementation choices, result interpretation, and academic claims are my own.

Signature: ______________________

Name: Graham Pellegrini

Date: ______________________

Contents

Plagiarism Declaration **i**

Contents **iii**

List of Figures **v**

List of Tables **vi**

List of Abbreviations **vii**

Glossary of Symbols **1**

1 Background **1**
1.1 Visuomotor policies 1
1.1.1 Behaviour cloning and compounding error 2
1.1.2 Action chunking 2
1.2 Rigid body pose and the SE(3) representation 3
1.3 Latent representations 4
1.4 Conditional variational autoencoders 5
1.5 Transformer networks and the attention mechanism 7
1.6 Self-supervised learning 8

2 Literature Review **10**
2.1 Action Chunking with Transformers (ACT) 10
2.2 Diffusion Policy 11
2.3 World models 13
2.3.1 World models as policy evaluators 13
2.3.2 Latent predictive architectures 15
2.4 Offline evaluation 20

3 Methodology **23**
3.1 Dataset and Experimental Setup 23
3.1.1 EgoDex: source and selection rationale 23
3.1.2 Windowing protocol 24

3.1.3 Training splits and test set . . . 25
3.2 Three-Branch Experimental Framework . . . 26
3.3 Policy Variants . . . 28
3.4 World Model Families . . . 29
3.5 Fair Experimental Protocol . . . 31
3.5.1 Training exposure budget . . . 31
3.5.2 Evaluation contract . . . 32
3.6 Evaluation Metrics . . . 32
3.6.1 Offline trajectory prediction setting . . . 32
3.6.2 Primary metrics . . . 32
3.6.3 Candidate selection comparison . . . 34
3.7 Evaluation Scope and Limitations . . . 34

4 Implementation **36**
4.1 Repository and execution model . . . 36
4.2 Data pipeline and windowing . . . 36
4.2.1 Index construction . . . 36
4.2.2 Incremental training curriculum . . . 37
4.3 Policy training . . . 37
4.4 World model training . . . 38
4.4.1 V-JEPA2-AC: extraction and transition head . . . 38
4.4.2 LeWM: end-to-end training . . . 39
4.4.3 DexWM: from-scratch training . . . 40
4.4.4 V-JEPA2 (no action conditioning): ablation control . . . 41
4.4.5 PointWorld: out-of-domain proxy . . . 41
4.5 Convergence study and universal training budget . . . 42
4.5.1 Loss curves and plateau detection . . . 42
4.5.2 Universal budget decision . . . 43
4.6 B2 margin threshold calibration . . . 45
4.7 Guard evaluation . . . 45
4.7.1 Branch 1: policy-only baseline . . . 45
4.7.2 Branch 2: V-JEPA2-AC veto guard . . . 45
4.7.3 Branch 3: world-model argmax . . . 46
4.7.4 Manifest and checkpoint design . . . 46
4.8 Evaluation safeguards . . . 46
4.9 Implementation challenges and resolutions . . . 47
4.10 Auxiliary modules . . . 49

List of Figures

Figure 1.1 A visuomotor policy maps visual observations to motor actions. The input is a camera frame showing the hand and object, and the output is a proposed action for how the hand should move next. 1
Figure 1.2 Compounding error in behaviour cloning. The policy trajectory drifts progressively further away at each step. The growing gap between the two paths illustrates how small errors can accumulate over time [6]. 2
Figure 1.3 Translation for each arm can be represented by a movement along each of the three spatial axis [11]. 3
Figure 1.4 Rotation around the y-axis for each arm. The same circular movement can be imagined around the x and z axes; the total orientation of the wrist is the combined result of rotations around all three [11]. 4
Figure 1.5 Example latent-space plot showing how structurally similar items cluster together in the learned representation. Structural similarity here refers to items that share shape, geometry, and likely physical behaviour [13]. 5
Figure 1.6 Standard Variational Autoencoder (VAE) (a) and conditional Conditional Variational Autoencoder (CVAE) (b) encoder-decoder pipelines [16]. . . 6
Figure 1.7 VAE encoder-decoder architecture and two-part training loss (reconstruction and regularisation) [17]. 6
Figure 1.8 Transformer attention block: each token queries all others with Q, gathers weighted values V via keys K, and the weighted sum yields the updated output token [20]. 7
Figure 1.9 Three machine learning paradigms: supervised learning (explicit labels), reinforcement learning (reward signal), and self-supervised learning (unlabelled data provides its own supervision signal). The figure labels this third paradigm as unsupervised learning; the two terms are used interchangeably in this context [22]. 8

Figure 2.1 Architecture of the Action Chunking with Transformers (ACT) policy [7]. 11
Figure 2.2 Diffusion Policy: general formulation (a), Convolutional Neural Network (CNN) variant (b), transformer variant (c) [8]. 12
Figure 2.3 Classic closed-loop control (a) versus CLOVER's closed-loop visuomotor control (b) [28]. 14

Figure 2.4 Three Self-Supervised Learning (SSL) prediction paradigms: joint-embedding (a), generative (b), and joint-embedding predictive (c) [21]. 15
Figure 2.5 Video Joint Embedding Predictive Architecture 2 (V-JEPA2) post-training pathways from internet-scale backbone to specialised applications [31]. . . . 16
Figure 2.6 Standard V-JEPA2 (left) versus action-conditioned Action-Conditioned Video Joint Embedding Predictive Architecture 2 (V-JEPA2-AC) (right) [31]. . 17
Figure 2.7 High-level layout of the DexWM world model pipeline, illustrating how predicted keypoints are incorporated alongside the encoder, Conditional Diffusion Transformer (CDiT) predictor, and decoder stages [33]. 18
Figure 2.8 LeWM training pipeline: encoder, dynamics predictor, and SIGReg regulariser [34]. 19

Figure 3.1 Fixed-stride windowing scheme (`obs16_pred16_s128`) used as the canonical training and evaluation index. Author-generated schematic. 25
Figure 3.2 State-change-targeted sampling pipeline yielding one frozen window per episode. Author-generated schematic. 25
Figure 3.3 Three-branch design space. B1 uses the policy's own preference alone; B2 adds a world-model veto constrained by a margin threshold; B3 gives the world model full selection authority over the frozen candidate pool produced at B2 time. The policy is not re-run in B3. 28

List of Tables

Table 2.1 Average consecutive sub-task completions (`avg_len`) on a real-world multi-step manipulation suite. Higher is better. CLOVER's closed-loop feedback via world model prediction nearly doubles the completion depth of the strongest open-loop baseline. Data from [28]. 14

Table 3.1 EgoDex splits under the `obs16_pred16_s128` windowing contract. Episode and window counts are derived from the canonical index. 26

Table 3.2 World model families and their methodological roles in the benchmark. 30

Table 4.1 Validation loss at each training scale for all seven model families. Losses are not comparable across rows (each family uses a different objective). The relevant signal is each family's trend across scales. The universal budget of 100k is highlighted. 43

Table 4.2 Plateau budget per primary model family and reasoning. The universal budget is $E = 100{,}000$ effective items. 44

List of Abbreviations

ACT Action Chunking Transformer.

ALOHA A Low-cost Open-source Hardware System for Bimanual Teleoperation.

BC Behaviour Cloning.

CDiT Conditional Diffusion Transformer.

CNN Convolutional Neural Network.

CVAE Conditional Variational Autoencoder.

DDIM Denoising Diffusion Implicit Models.

DDPM Denoising Diffusion Probabilistic Models.

DexWM Dexterous Manipulation World Model.

DiT Diffusion Transformer.

DP Diffusion Policy.

GATO Generalist Agent.

HDF5 Hierarchical Data Format 5.

I-JEPA Image Joint Embedding Predictive Architecture.

JEPA Joint Embedding Predictive Architecture.

LeWM LeWorldModel.

RMSE Root Mean Square Error.

SSL Self-Supervised Learning.

V-JEPA2 Video Joint Embedding Predictive Architecture 2.

V-JEPA2-AC Action-Conditioned Video Joint Embedding Predictive Architecture 2.

VAE Variational Autoencoder.

ViT Vision Transformer.

VLM Vision Language Model.

1 Background

This chapter assumes familiarity with the basic mechanics of a neural network and introduces six concepts central to this work: visuomotor policies, rigid-body pose in SE(3), latent representations, conditional VAEs, transformer attention, and self-supervised learning. These foundations support understanding of the specific architectures in Chapter 2 and underpin the candidate selection mechanism and world model evaluator design examined in Chapter 3.

1.1 Visuomotor policies

A visuomotor policy is a learned function that takes a visual observation as input and produces an action or sequence of future actions as output. In a manipulation setting, the visual observation is typically a single camera frame showing the hand and the object being manipulated, and the action describes how the hand should move next. The policy is called visuomotor because it connects vision directly to motor output, without requiring a hand-crafted intermediate representation of what the scene contains [1].

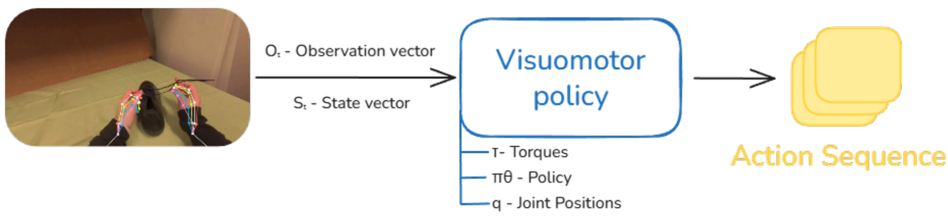

Figure 1.1 A visuomotor policy maps visual observations to motor actions. The input is a camera frame showing the hand and object, and the output is a proposed action for how the hand should move next.

Learning this mapping from data is appealing because useful hand behaviour depends on subtle visual cues that are difficult to specify by hand. The timing of a reaching movement, the grip shape adopted before contact, and the postural adjustments made in response to object geometry all depend on information visible in the camera image but difficult to encode as explicit rules [2–4]. A visuomotor policy learns these associations directly from recorded demonstrations, capturing the relationship between what the camera sees and what the hand does without requiring the programmer to describe it explicitly.

1.1.1 Behaviour cloning and compounding error

The most direct way to train a visuomotor policy from demonstrations is Behaviour Cloning (BC) [1]. Here an observed frame predicts the action taken by the expert at that moment, and the policy is trained to minimise the difference between its predicted action and the recorded expert action across all frames in the dataset [5]. The problem arises when a small prediction error causes the hand to move to a state that is slightly different from any state in the training data. This results in a compounding error: as the policy encounters increasingly unfamiliar states, its prediction errors grow larger over time.

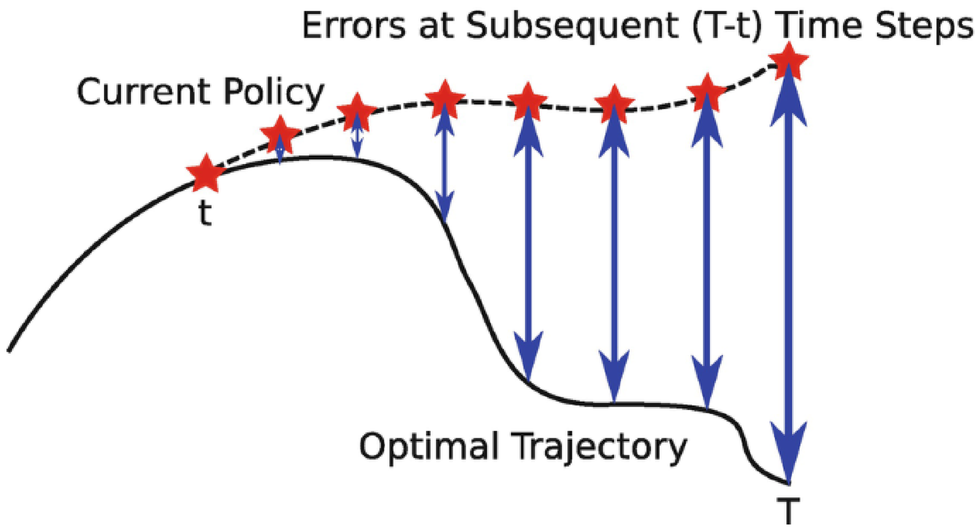

Figure 1.2 Compounding error in behaviour cloning. The policy trajectory drifts progressively further away at each step. The growing gap between the two paths illustrates how small errors can accumulate over time [6].

1.1.2 Action chunking

Action chunking addresses the compounding error problem by changing what the policy predicts. Instead of predicting a single next action at each step, the policy predicts a short sequence of future actions in one forward pass. This sequence is called an action chunk [7, 8]. This is why Figure 1.1 shows the action sequence as a stack of multiple action blocks. By committing to a chunk of k future steps, where k is a hyperparameter set per task, the policy makes one prediction decision every k frames rather than one per frame. With fewer decision points, there are fewer opportunities for errors to accumulate, and the hand stays closer to the training distribution for longer [9].

1.2 Rigid body pose and the SE(3) representation

To predict how a hand will move, a system needs a precise way to describe where the hand is and which way it is pointing. These two quantities together are called the pose of a rigid body. For the human wrist, pose is fully described by its position in space and its orientation relative to a fixed reference frame [10].

Translation – Position is represented as a vector of three numbers, one for each spatial dimension: how far the wrist is to the right or left, how far forward or backward, and how far up or down [2]. This vector **t** $= [t_x,\ t_y,\ t_z]^\top$ is straightforward to predict with no special constraints on the output.

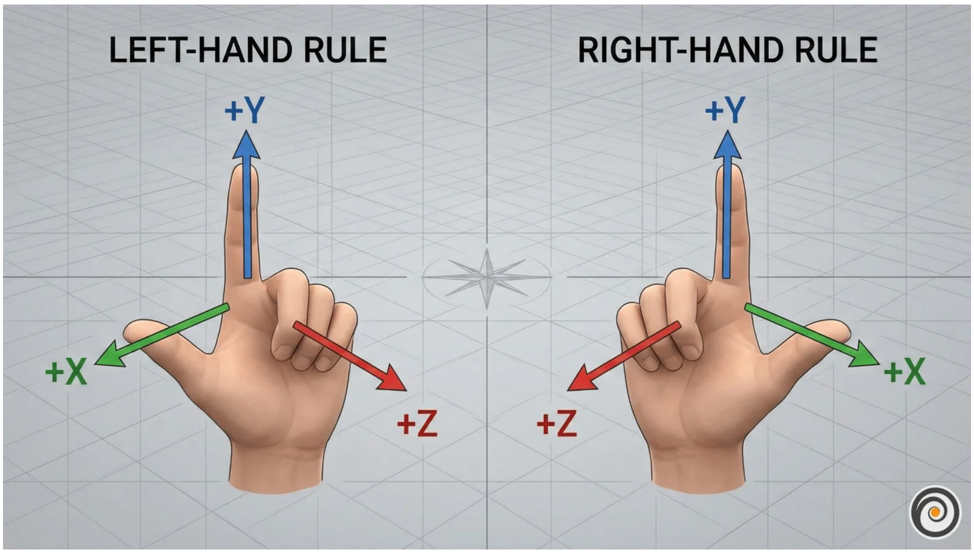

Figure 1.3 Translation for each arm can be represented by a movement along each of the three spatial axis [11].

Rotation – Orientation requires more care. A common representation uses three angles known as Euler angles, one per axis. These are intuitive but problematic for learning. At certain configurations the axes align and the system loses a degree of freedom, a singularity called gimbal lock. A subtler problem is discontinuity: a wrist rotated 359 degrees and one rotated 1 degree are almost physically identical, yet their angle values are numerically far apart. A network minimising prediction error will produce large errors near these boundaries even when the true motion is smooth [12].

A rotation matrix R avoids both issues: small physical rotations produce small numerical changes in the matrix entries, and there are no singularities.

The homogeneous transform – Translation and rotation are combined into a single 4×4 matrix, the standard representation of a pose in the special Euclidean group SE(3):

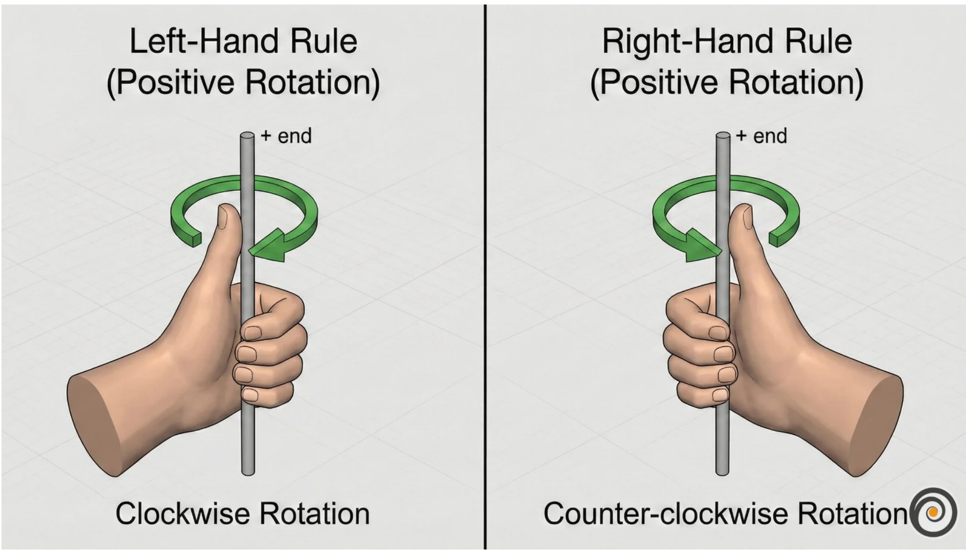

Figure 1.4 Rotation around the y-axis for each arm. The same circular movement can be imagined around the x and z axes; the total orientation of the wrist is the combined result of rotations around all three [11].

$$T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \tag{1.1}$$

1.3 Latent representations

A latent representation is a learned internal description of an observation. An image starts as a grid of pixel values, where each pixel stores three or four channel intensities (typically red, green, and blue, with an optional transparency value). A model does not usually reason with these raw numbers directly. Instead, it transforms them through several layers into a new set of values that no longer correspond to individual pixel coordinates or colours. These new values form a compact encoding that captures patterns in the image such as shape, position, motion, and spatial structure. The collection of these encodings is often called the latent space. Figure 1.5 shows an example in which similar items cluster together in the learned representation.

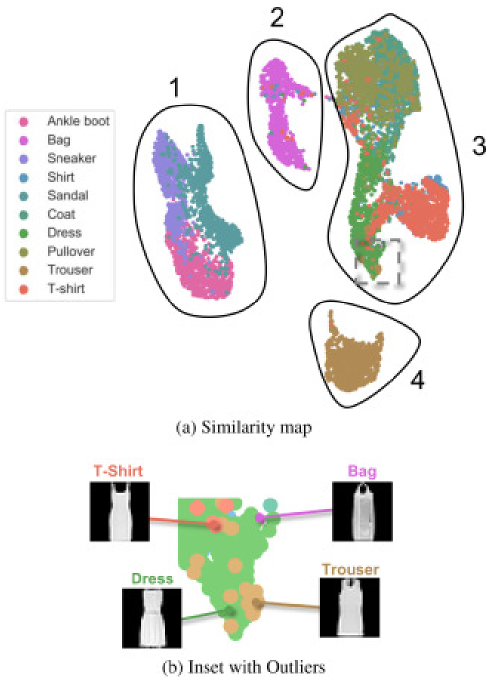

Figure 1.5 Example latent-space plot showing how structurally similar items cluster together in the learned representation. Structural similarity here refers to items that share shape, geometry, and likely physical behaviour [13].

The purpose of this transformation is to keep task-relevant structure while discarding unnecessary visual detail. For a manipulation scene, a useful latent state might encode hand position, object pose, contact progression, or motion trend without preserving every texture or lighting variation. A latent state is therefore not directly interpretable in the way an image is, but it can still preserve the information needed to reason about what happens next.

This is why latent prediction is often a better fit than pixel-level prediction for high-dimensional video observations. Predicting the next frame at the pixel level requires a model to account for every visual detail of the scene, including lighting changes, texture variation, and background movement that carry no information about task-relevant dynamics. A latent predictor instead operates on compact embeddings and concentrates predictive capacity on the structure that determines what actually happens next [14]. This design trade-off motivates the world model architectures reviewed in Chapter 2.

1.4 Conditional variational autoencoders

A VAE maps an input to a distribution of latent vectors described by a mean μ and standard deviation σ, rather than a single point. Sampling different latent vectors lets the decoder produce different but plausible outputs rather than one fixed answer. Figure 1.6a shows this standard encoder-latent-decoder flow [15].

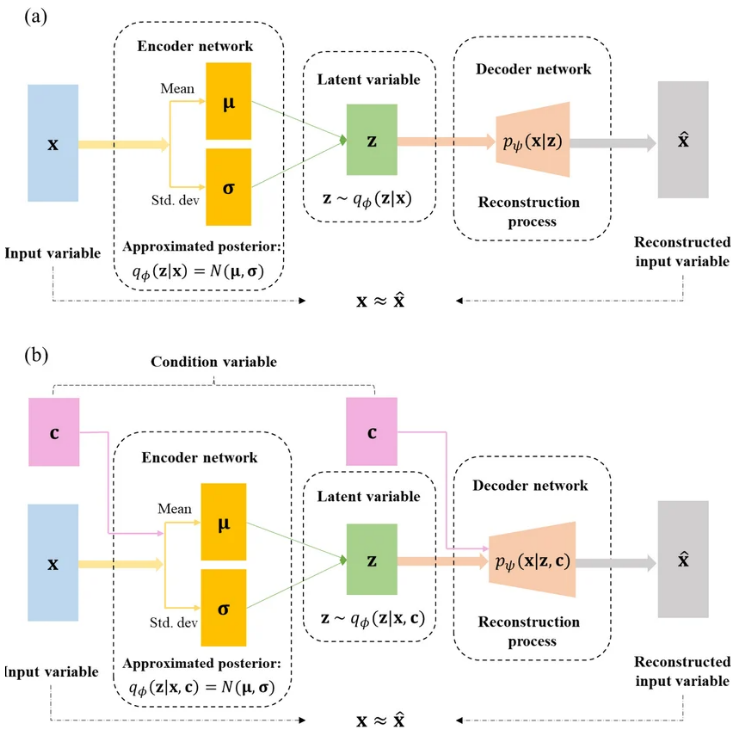

Figure 1.6 Standard VAE (a) and conditional CVAE (b) encoder-decoder pipelines [16].

For the policy to be trainable end-to-end, gradients must flow from the action prediction loss all the way back to the encoder. Sampling z directly from a distribution blocks this path. The reparameterisation trick resolves this by writing $z = \mu + \sigma\epsilon$, where ϵ is standard normal noise drawn independently of the learned parameters. Figure 1.7 shows the full encoder-latent-decoder pipeline alongside its training loss: a reconstruction term that rewards accurate output and a regularisation term that keeps the latent distribution close to a standard Gaussian, together forming the evidence lower bound [15].

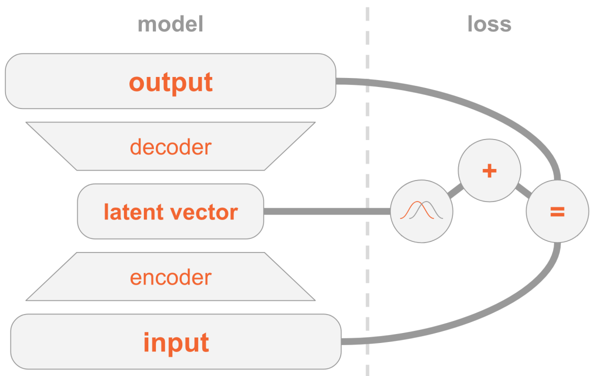

Figure 1.7 VAE encoder-decoder architecture and two-part training loss (reconstruction and regularisation) [17].

A CVAE adds an extra conditioning input to both encoder and decoder, shown in Figure 1.6b. In a visuomotor policy, this condition is the current observation frame, so the latent variable only needs to capture how the future action varies within that scene [18]. The Action Chunking Transformer (ACT) architecture uses this to sample multiple latent vectors from one observation and decode each into a candidate action chunk [7].

1.5 Transformer networks and the attention mechanism

A transformer processes a sequence as a set of tokens. A token can represent a word, an image patch, a timestep in an action sequence, or any vector produced by an earlier model component. The architectural idea needed here is simple: each token is updated by looking across the other tokens and deciding which of them are most relevant. This relevance-weighted exchange of information is called attention [19].

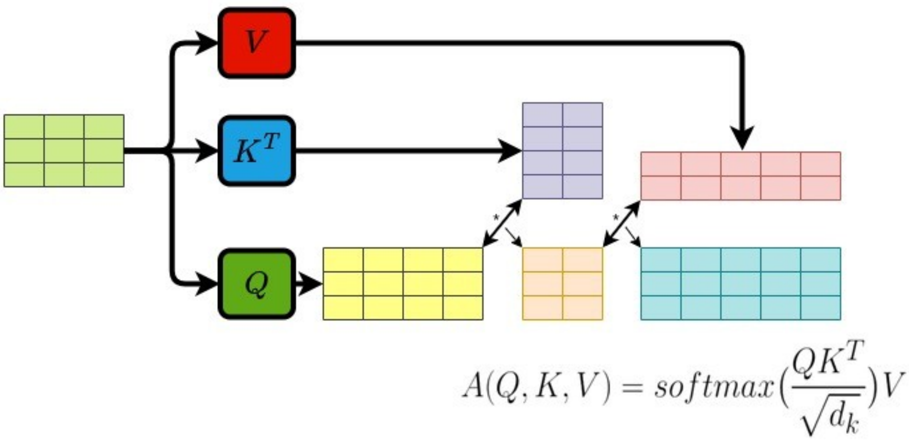

Figure 1.8 Transformer attention block: each token queries all others with Q, gathers weighted values V via keys K, and the weighted sum yields the updated output token [20].

Attention forms three learned views of every token:

- **Query** (Q) – what this token is looking for.
- **Key** (K) – what another token contains.
- **Value** (V) – what another token contributes if attended to.

Figure 1.8 shows how each token (highlighted) sends its query outward to every other token, receives weighted values in return, and produces an updated output. The bidirectional arrows indicate that every token attends to every other token simultaneously, and the crossing arrow beneath represents the weighted sum that combines those values into the final output. For each token, the transformer compares its query with the keys of all tokens. The resulting scores become weights, and those weights are used to combine the corresponding values. This operation is usually written as [19]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \tag{1.2}$$

Transformers usually repeat this operation in several heads and add positional information so the model knows where each token sits in the sequence. Multiple heads allow different relationships to be represented at the same time, while positional information preserves order [19].

1.6 Self-supervised learning

Machine learning methods can be grouped by how they receive feedback during training, as illustrated in Figure 1.9. Supervised learning trains on data with explicit labels: the correct answer is provided for every input, and the model learns to reproduce it. BC from Section 1.1.1 is a direct example: the expert demonstration acts as the label. Reinforcement learning instead receives feedback through a reward signal after taking actions in an environment; the model learns which state-action sequences lead to good outcomes. SSL sits between these two: it trains on unlabelled data by having the data itself provide the supervision signal. Part of the input is hidden, and the model learns to predict it from the remainder [21].

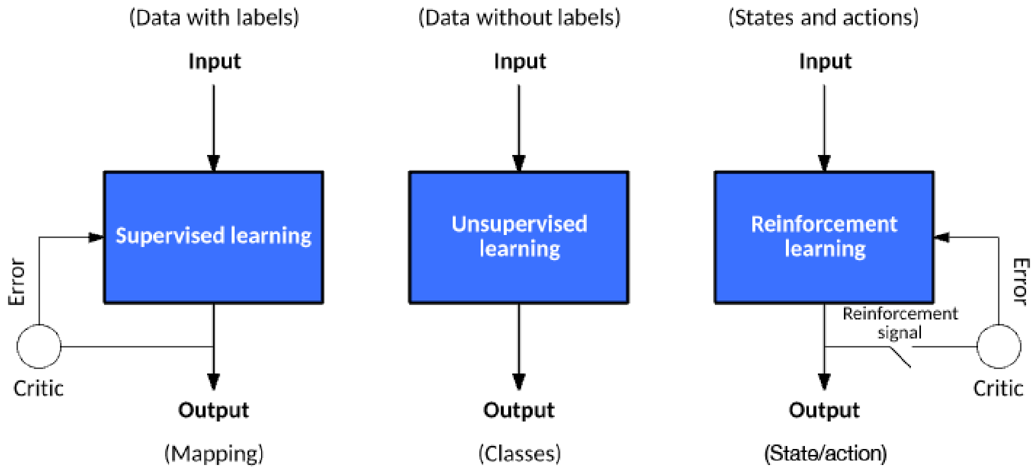

Figure 1.9 Three machine learning paradigms: supervised learning (explicit labels), reinforcement learning (reward signal), and self-supervised learning (unlabelled data provides its own supervision signal). The figure labels this third paradigm as unsupervised learning; the two terms are used interchangeably in this context [22].

Applied to video, SSL takes the following form: given a sequence of observed frames, hide one or more future frames and ask the model to predict them from the frames that came before. No human annotation is required because the video recording itself provides the hidden target.

Predicting raw pixels runs into a problem described in Section 1.3: when the future is ambiguous the model cannot commit to a single outcome, so it predicts the average of all possibilities and produces a blurred image that does not match any real frame. Predicting in latent space avoids this. Instead of reconstructing every pixel, the model predicts the compact embedding of the future frame. Background motion and fine texture are discarded at the encoding step, so the predictor focuses on the structure that determines what actually happens next [14]. When the predictor is also given the action the agent intends to take, the result is a world model: a learned function that predicts how the scene embedding will change in response to a proposed action. The architectures that build on this idea are reviewed in Chapter 2.

2 Literature Review

This chapter surveys the literature motivating the three central design choices of this study's benchmark: that a specialist imitation policy is a strong base for bimanual manipulation, that a world model can usefully evaluate candidate action chunks in an offline benchmark, and that latent predictive learning is the right approach for building such an evaluator. Sections 2.1 and 2.2 review the two candidate imitation policies, ACT and Diffusion Policy, examining both the challenges they address and the trade-offs between them. Section 2.3 introduces world models as learned environment predictors, surveys the evaluator architectures most relevant to this role, and covers the three latent predictive architectures implemented in the benchmark, together with a negative control. Section 2.4 covers offline evaluation frameworks and their validity as proxies for real-world performance. The chapter closes with a summary of the literature gap that the benchmark addresses. The chapter assumes familiarity with the technical foundations in Chapter 1; world models are introduced here.

2.1 Action Chunking with Transformers (ACT)

ACT was introduced by Zhao et al. to address the compounding error and multimodal distribution challenges that make dexterous bimanual manipulation demanding for imitation learning [7]. On the A Low-cost Open-source Hardware System for Bimanual Teleoperation (ALOHA) bimanual platform it achieved success rates of up to 90% on precision tasks such as threading a cable tie, inserting a battery, and opening a translucent bag, outperforming behaviour cloning baselines and Generalist Agent (GATO) by substantial margins [7]. In SE(3) wrist-pose prediction specifically, small deviations in predicted position or orientation alter contact geometry in ways that cascade into grasp failure [1], and a policy trained with a naïve regression loss produces averaged predictions that may not represent any plausible trajectory when the demonstrated distribution is multimodal [8]. ACT's CVAE architecture addresses both of these failure modes directly.

ACT addresses both challenges through the use of a CVAE design (Section 1.4). The respective CVAE has an encoder-decoder structure depicted in Figure 2.1, where the encoder compresses an action sequence and joint observation into a style variable z that captures the multimodal distribution of plausible futures, and the decoder takes visual observations and joint positions, together with z, through a transformer encoder, and predicts a sequence of future action steps as an action chunk with a transformer decoder. The encoder is discarded at test time. In the original formulation, z is set to the mean of the prior (zero) to produce a single deterministic action chunk per observation [7]. When a pool of candidate chunks is required, multiple values of z are sampled independently from the prior and each is decoded into a separate candidate, all conditioned on the same observation.

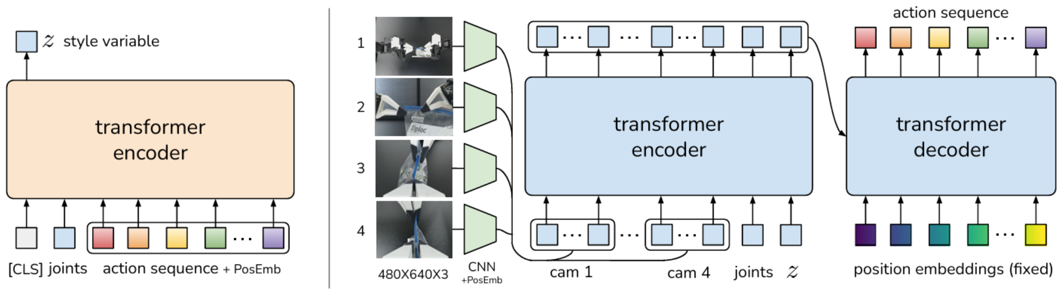

Figure 2.1 Architecture of the Action Chunking with Transformers (ACT) policy [7].

At inference time, the CVAE decoder produces each candidate action chunk by decoding an independently sampled z vector. A temporal ensembling technique is then applied: consecutive chunks overlap in their predicted time horizons, and a recency-weighted average is taken across overlapping predictions to produce the final action. This averaging smooths the executed motion and reduces jerkiness caused by abrupt chunk transitions. The result is a structured pool of plausible futures, all conditioned on the same observation, from which the ensembled output is drawn [7].

ACT carries known limitations alongside these strengths. The Gaussian prior on the style variable z constrains how faithfully the CVAE can represent highly multimodal action distributions: at transitions where qualitatively different trajectories are equally plausible, the latent space can average across modes rather than sample cleanly from each one, producing trajectories that are plausible in aggregate but imprecise at the most ambiguous decision points [8]. Chunk length is a fixed hyperparameter that must be tuned per task; longer chunks improve smoothness but commit the policy to a trajectory over a horizon that may no longer be appropriate as the scene evolves. Temporal ensembling introduces a recency-weighted lag that can slow the policy's response to sudden environmental changes [7]. These properties are well-documented in follow-on work comparing ACT with more expressive policy classes, and they motivate the inclusion of Diffusion Policy as the second candidate in the benchmark.

2.2 Diffusion Policy

Diffusion Policy offers a principled alternative approach to the same compounding error and multimodality challenges. Its core concept relies on starting from a noise distribution and iteratively refining action predictions through a diffusion process. By gradually denoising the predictions, Diffusion Policy captures complex multimodal distributions effectively, producing plausible trajectories that align with the demonstrated data [8]. Figure 2.2 illustrates the policy overview:

- a) General formulation: at timestep t, the policy takes the latest T steps of observation data O_t as input and outputs T steps of actions A_t.
- b) CNN-based Diffusion Policy: FiLM (Feature-wise Linear Modulation) [23] conditioning of the observation feature O_t is applied to every convolution layer, channel-wise. Starting from K_t drawn from Gaussian noise, the output of noise-prediction network ϵ_θ is subtracted, repeating K times to get A_t^0, the denoised action sequence.
- c) Transformer-based Diffusion Policy: the embedding of observation O_t is passed into a multi-head cross-attention layer of each transformer decoder block. Each action embedding is constrained to only attend to itself and previous action embeddings (causal attention) using the attention mask illustrated [24, 25].

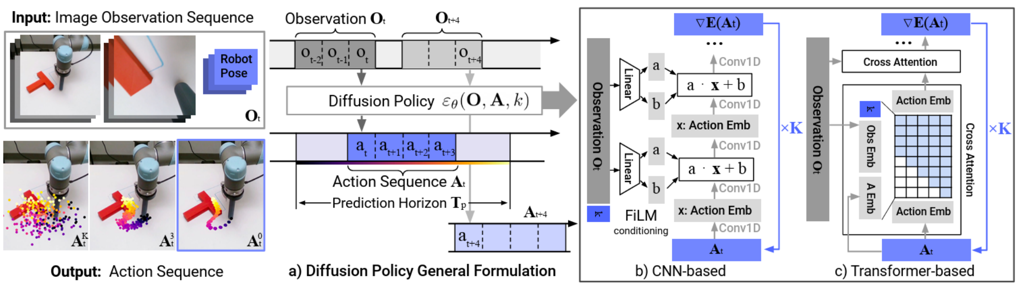

Figure 2.2 Diffusion Policy: general formulation (a), CNN variant (b), transformer variant (c) [8].

Chi et al. report that Diffusion Policy outperforms ACT on tasks with high action multimodality (such as multi-step object rearrangement and cup placement under visual ambiguity) while matching it on simpler reaching tasks [8]. The advantage arises because iterative denoising explores the full action distribution at each step rather than committing to a single latent mode as the CVAE prior must, making it more robust to the mode-averaging failure that affects ACT at ambiguous transitions.

The main limitation of Diffusion Policy is inference cost. Generating a single action chunk requires multiple sequential network evaluations: standard Denoising Diffusion Probabilistic Models (DDPM) variants use on the order of 100 denoising steps compared with the single forward pass of ACT's decoder, which substantially reduces the achievable control frequency [8]. Efficient transformer variants address this through progressive noise scheduling and parameter sharing, recovering much of the inference speed at added architectural complexity [25]. Both policies are included in the benchmark to span this representational trade-off: ACT's structured latent sampling is computationally efficient but expressively constrained at high multimodality, while Diffusion Policy's iterative denoising is more expressive but more demanding at inference. Measuring the world model's effect across both allows the benchmark to determine whether evaluator gains are consistent across this axis.

2.3 World models

A world model is a learned predictor of how a system evolves over time. Given the current state and a proposed action, it estimates the state that is likely to follow. In a manipulation setting, the state may be a raw image, a latent representation of the scene (Section 1.3), or a structured description of hand and object configuration. Regardless of the representation chosen, the underlying principle is the same: the model internalises the dynamics of the environment and uses them to project forward from any given state [26].

World models become especially useful when a pool of candidate actions must be evaluated before any one is committed to. Each candidate is rolled forward through the model, producing a predicted future state; those predictions are ranked by predicted cost or consistency. The quality of this ranking depends heavily on the chosen representation: pixel-space world models must reconstruct every visual detail and suffer from mode-averaging at multimodal transitions, while models that operate in the compact learned latent space introduced in Section 1.3 can concentrate predictive capacity on the dynamics that matter for task outcomes [14]. The following subsections survey the literature supporting this evaluator design and describe the three world model architectures implemented in the benchmark.

2.3.1 World models as policy evaluators

WorldGym demonstrates that a learned world model used as a simulated environment for policy evaluation produces rankings that align with real-world task success with meaningful fidelity [27]. Candidate policies are evaluated not by deploying them physically but by rolling them out inside the model's predicted environment, reducing the cost of policy comparison to a forward pass. CLOVER extends this to closed-loop control, as shown in Figure 2.3. A visual planner generates a sequence of sub-goal frames; the executor measures the embedding-space error between the predicted sub-goal and the current observation. When the error exceeds a threshold, the system triggers an adaptive transition and replans rather than committing to the original trajectory. The feedback loop, absent from open-loop imitation policies, is the mechanism that translates world model prediction accuracy into task completion gain [28]. Together, WorldGym and CLOVER establish that world model rankings are informative both before and during execution.

The broader question of whether such evaluator rankings transfer to physical outcomes, and the qualifications around out-of-distribution reliability, are examined in Section 2.4.

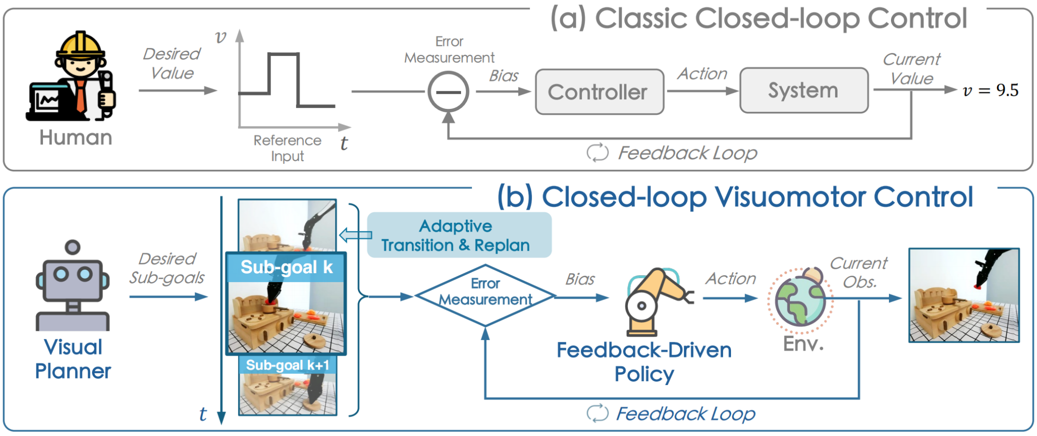

Figure 2.3 Classic closed-loop control (a) versus CLOVER's closed-loop visuomotor control (b) [28].

The practical value of this evaluator loop is measurable. Table 2.1 reproduces CLOVER's real-world evaluation on a multi-step robot manipulation suite, reporting the average number of consecutive sub-tasks completed (`avg_len`) for representative baselines and CLOVER [28].

Table 2.1 Average consecutive sub-task completions (`avg_len`) on a real-world multi-step manipulation suite. Higher is better. CLOVER's closed-loop feedback via world model prediction nearly doubles the completion depth of the strongest open-loop baseline. Data from [28].

Method	avg_len
ACT [7]	0.6
R3M (pretrained visual repr.) [29]	0.7
RT-1 [30]	1.1
CLOVER (closed-loop)	**2.1**

Alongside the task completion gain, CLOVER reduces embedding-space prediction Root Mean Square Error (RMSE) relative to open-loop baselines by correcting the policy's course when the world model detects a divergence from the expected trajectory, demonstrating that prediction accuracy and task success move together under the evaluator framing.

2.3.2 Latent predictive architectures

The energy-based framework of LeCun and Dawid establishes why prediction should operate in latent rather than pixel space [14]: the same representational argument introduced in Section 1.3 applies directly to the prediction target, favouring compact embeddings over pixel reconstructions that force the model to account for irrelevant visual detail and average over all plausible continuations at ambiguous transitions.

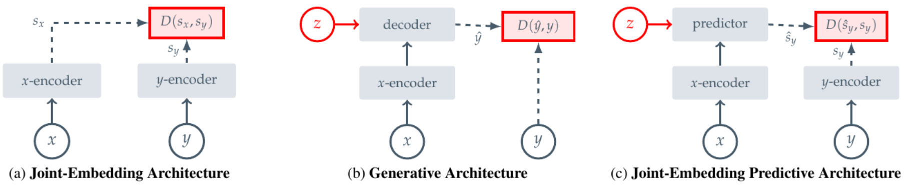

Figure 2.4 Three SSL prediction paradigms: joint-embedding (a), generative (b), and joint-embedding predictive (c) [21].

Image Joint Embedding Predictive Architecture (I-JEPA) establishes the joint-embedding predictive architecture as a viable approach to learning rich visual representations without any pixel reconstruction objective [21]. Panel (c) of Figure 2.4 shows the data flow directly: a context block x from the image is passed through the context encoder to produce latent tokens z_x. A lightweight ViT predictor then takes z_x together with positional tokens indicating where the masked target regions y are located, and outputs predicted latent representations $\hat{s}_y$ for those regions. A separate target encoder, whose weights are a slow exponential moving average of the context encoder, independently encodes the actual target pixels into ground-truth latents s_y. The loss MSE$(\hat{s}_y, s_y)$ is computed entirely in latent space: no decoder, no pixel generation, no reconstruction of lighting or texture. The outcome is a context encoder whose representations capture semantically meaningful structure (object shape, spatial layout, and scene dynamics) because that is all the predictor ever needs to get the target embeddings right. Training is approximately 2.5 times faster than pixel-reconstruction methods as a result. V-JEPA2 extends this same joint-embedding predictive architecture from images to video sequences.

Four architectures are covered in the following subsections, each representing a distinct representational strategy. V-JEPA2 and V-JEPA2-AC share the same pretrained video backbone: V-JEPA2 is the internet-scale pretrained encoder, while V-JEPA2-AC extends it with an action-conditioned transition head and is the form used directly in the benchmark. DexWM uses a pretrained image backbone with specialised hand-geometry supervision. LeWorldModel (LeWM) learns all representations from scratch on the task data itself.

V-JEPA2

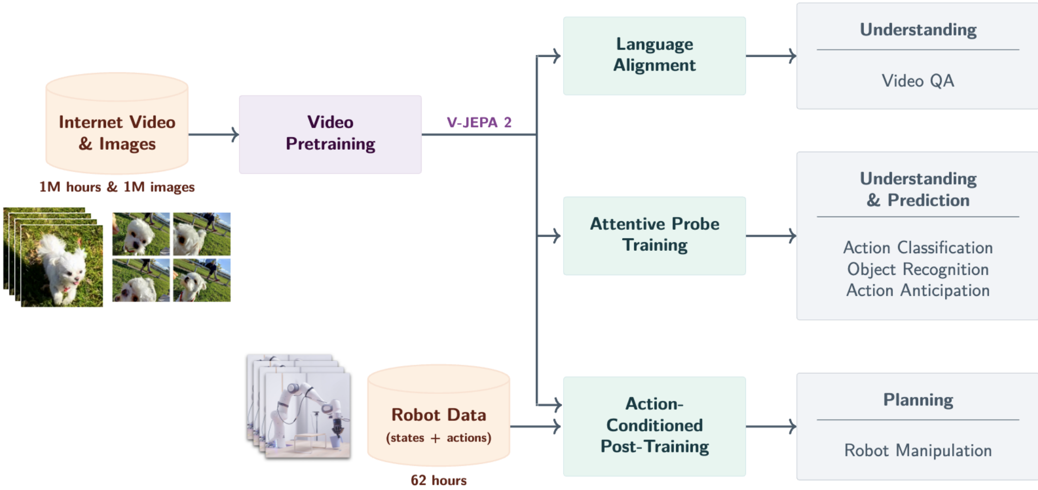

Figure 2.5 V-JEPA2 post-training pathways from internet-scale backbone to specialised applications [31].

V-JEPA2 extends the I-JEPA framework from static images to video, pretraining on over one million hours of internet video [31]. The pretraining shifts from spatial to spatio-temporal masking: the context encoder receives past observation frames, and the predictor must recover future latent states. The training objective combines teacher-forcing with multi-step autoregressive rollout losses, encouraging accurate short-horizon predictions and consistent long-horizon trajectory representations. The result is a frozen ViT-Large encoder, approximately 300 million parameters, that encodes physical priors about hand and object interactions acquired from internet-scale pretraining. As shown in Figure 2.5, the pretrained backbone supports three post-training pathways: Language Alignment, which grounds visual embeddings in natural language; Attentive Probe, which adds a lightweight classification head; and Action Conditioning, which extends the predictor to take robot actions as input and forms the basis of V-JEPA2-AC.

V-JEPA2-AC

The frozen ViT-Large encoder from V-JEPA2 serves as the backbone of V-JEPA2-AC, producing a fixed latent embedding for each observation frame with no gradient updates to the backbone weights. A lightweight action-conditioned transition head is trained on top of these stored embeddings: it takes the current frame latent together with a proposed action chunk (robot joint states and end-effector poses) as the conditioning signal z, and predicts the latent representation of the future frame. Candidate action chunks are then scored by comparing the predicted future embedding against the actual observed future embedding under an L1 objective. This design, shown on the right of Figure 2.6, separates what the network must learn into two parts: the frozen encoder supplies rich visual features that generalise from internet-scale pretraining, and the small trained head learns only how actions propagate through those features.

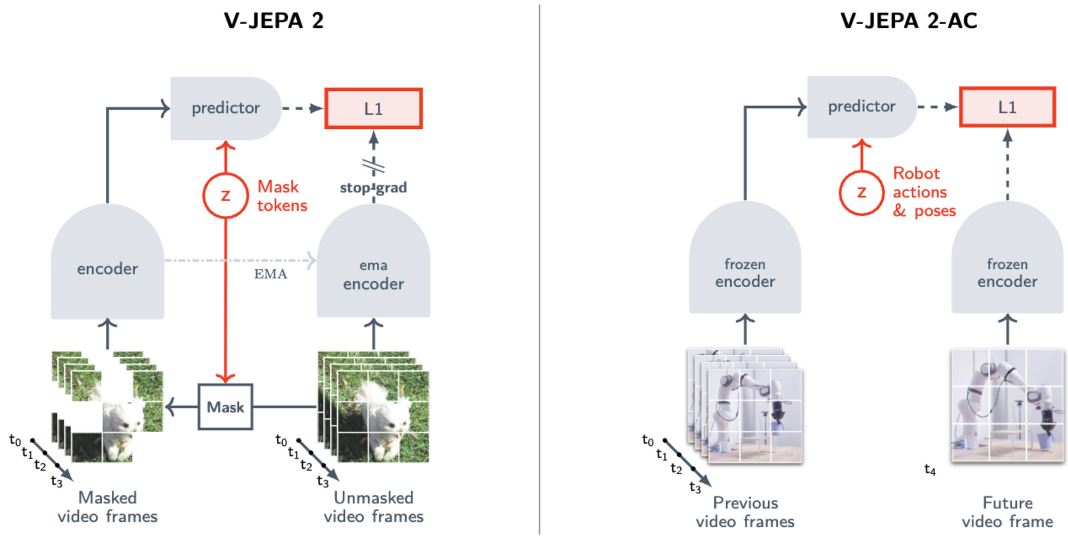

Figure 2.6 Standard V-JEPA2 (left) versus action-conditioned V-JEPA2-AC (right) [31].

An important qualification is raised by Tsagkas et al., who show that pretrained visual representations can encode spatial plausibility well while providing a weaker signal for fine-grained temporal dynamics [32]. A frozen encoder trained on internet video may represent hand shape and position accurately at each individual frame while providing a less reliable signal about whether the predicted trajectory is kinematically consistent across the full chunk horizon. This temporal limitation provides context for the path shape results in Chapter **??**.

DexWM

DexWM [33] is a dexterous manipulation world model from Meta FAIR. As shown in Figure 2.7, its pipeline encodes each input frame with a frozen DINOv2 image encoder, producing a grid of spatial patch tokens rather than a single global embedding. These patch-level representations preserve fine-grained spatial information about hand positions and object locations, which gives the predictor strong generalisation across diverse environments and viewpoints. A CDiT predictor then takes the spatial latent tokens together with action and camera-motion inputs to predict the next latent state, and a lightweight decoder maps the result back to image and keypoint space. Where V-JEPA2-AC uses a video-pretrained backbone, DexWM uses a backbone pretrained on static images; the comparison therefore isolates the contribution of video-specific temporal pretraining to world model quality.

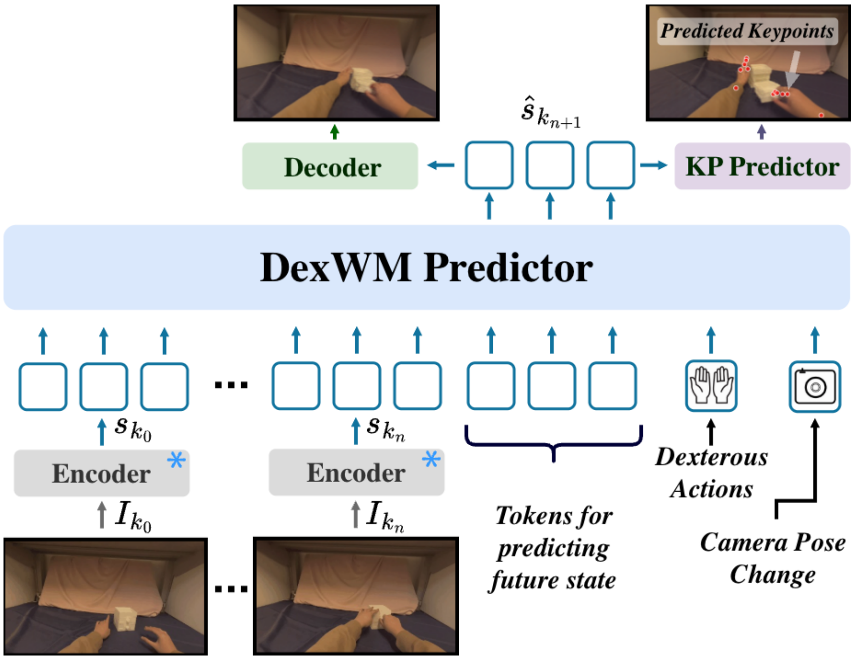

Figure 2.7 High-level layout of the DexWM world model pipeline, illustrating how predicted keypoints are incorporated alongside the encoder, CDiT predictor, and decoder stages [33].

A distinguishing feature of DexWM is the Hand Consistency (HC) loss: an auxiliary transformer head predicts fingertip and wrist heatmaps from the latent state, imposing a geometric prior that the predictor's latent trajectory must remain consistent with plausible hand configurations. This explicit hand-geometry supervision is absent from both V-JEPA2-AC and LeWM, whose latent spaces are shaped only by prediction error or general regularisation. Goswami et al. report a 50% improvement over Diffusion Policy on grasping and placing tasks, with an 83% zero-shot real-world grasping success rate under latent planning [33].

LeWorldModel

LeWM is a latent world model trained end-to-end from random initialisation on raw pixel observations [34]. Its architecture consists of two components: an encoder that maps each frame o_t to a compact latent vector z_t, and a predictor that models environment dynamics by estimating the next latent embedding $\hat{z}_{t+1}$ from z_t and the action a_t. The encoder is a ViT-Tiny with approximately 15 million parameters. Each frame is represented as a single 192-dimensional token rather than a spatial patch grid, which keeps the latent space compact and makes planning significantly faster than models that produce higher-dimensional patch representations.

As shown in Figure 2.8, the training pipeline consists of an encoder that maps each observation frame to a compact latent vector, a dynamics predictor that autoregressively estimates the next latent given the current embedding and action, and

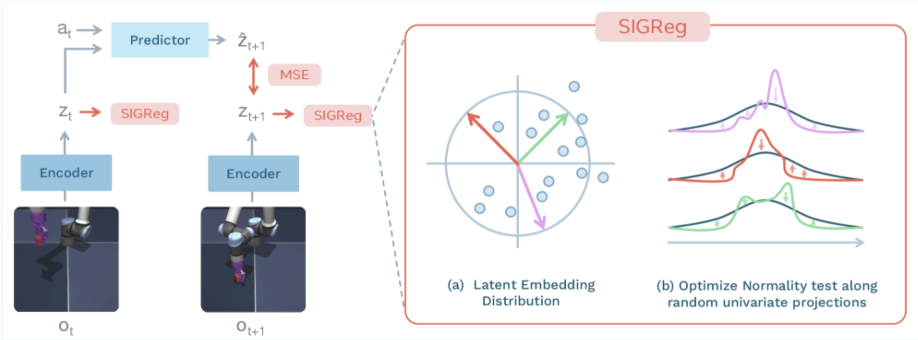

Figure 2.8 LeWM training pipeline: encoder, dynamics predictor, and SIGReg regulariser [34].

a SIGReg regulariser that enforces a Gaussian prior on the latent distribution by optimising normality along random univariate projections. The training objective reduces to two terms:

$$\mathcal{L}_{\text{LeWM}} = \mathcal{L}_{\text{pred}} + \lambda \, \text{SIGReg}(Z) \tag{2.1}$$

where $\mathcal{L}_{\text{pred}}$ penalises inaccurate next-embedding forecasts and $\text{SIGReg}(Z)$ stabilises end-to-end training without contrastive negative pairs or an exponential moving average target encoder [34]. This reduces the number of tunable loss hyperparameters from six, as required by prior Joint Embedding Predictive Architecture (JEPA)-style designs, to one.

DINO-WM, the closest comparable baseline, follows a similar latent prediction objective but uses a frozen DINOv2 backbone producing a grid of spatial patch tokens per frame [35]. Like DexWM, DINO-WM therefore carries the spatial resolution and generalisation benefits of DINOv2's patch-level representations, but at substantially higher computational cost. LeWM replaces the frozen backbone with a fully learned ViT-Tiny encoder that collapses each frame to a single 192-dimensional token, trading spatial resolution for training speed. In practice, LeWM can be trained on a single GPU in a few hours and achieves a 48$\times$ planning speedup over DINO-WM [34]. DexWM's CDiT predictor operating on full patch grids is significantly slower to train and requires substantially more GPU memory; this difference in training cost is expected to widen further at larger dataset scales and is quantified in Chapter 4. Despite its compact size, LeWM's latent space encodes physically meaningful structure: the model detects physically implausible state transitions and captures spatial information such as agent and object locations across frames.

The three-way comparison across V-JEPA2-AC, DexWM, and LeWM is therefore not simply an architecture comparison. It is a comparison between fundamentally different representational strategies: video-pretrained transfer (V-JEPA2-AC), image-pretrained transfer with geometric supervision (DexWM), and self-supervised task-data learning from scratch (LeWM). This distinction makes the comparison principled and is the basis for the experimental design in Chapter 3.

PointWorld

PointWorld is a large pretrained 3D world model for in-the-wild rigid arm manipulation [36]. Unlike the latent-space models described above, it operates directly in 3D geometry: scene state is lifted from RGB-D images into a full-scene point cloud, and robot actions are represented as the 3D point flows traced by robot surface geometry through forward kinematics. This embodiment-agnostic representation allows the model to reason over physical interaction geometry rather than learned abstract features.

Such a model is not designed to handle a 16-dimensional dexterous hand action space, where fine-grained finger articulation drives the manipulation outcome. The forward-kinematics representation assumes rigid arm geometry, making the model structurally mismatched when applied to dexterous bimanual hand motion.

2.4 Offline evaluation

A central concern for any offline benchmark is whether its metrics produce rankings that are meaningful proxies for real-world performance. Without this property, an offline evaluation framework reduces to a measure of in-distribution accuracy with no predictive validity for deployment.

At large scale, DreamerV3 demonstrates the effectiveness of latent-space candidate evaluation across diverse continuous control domains, including locomotion and manipulation [26]. DreamerV3 is an online reinforcement learning system, not an offline benchmark; it is included here because it provides large-scale empirical evidence that latent-space evaluation is an effective selection mechanism when properly calibrated, which motivates its use in the offline setting examined in this study. Its world model maintains a compact recurrent latent state that transitions under action conditioning; candidate trajectories are rolled out entirely inside this imagined space and scored by a learned value function, with no return to the real environment between evaluations. TD-MPC2 confirms the principle with a scalable planner that optimises a temporal-difference value function defined over predicted latent trajectories rather than observed states [37]. Both systems show that ranking candidate actions by predicted latent-space outcomes is competitive with direct online evaluation across a range of motor control benchmarks.

A critical qualification is that latent-space scoring becomes unreliable outside the training distribution. MOPO, MOReL, MBOP, and MOPP each establish that learned models produce inaccurate predictions for out-of-distribution states, requiring conservative candidate selection to avoid trajectory degradation [38–41]. None of these works addresses dexterous manipulation specifically; they benchmark locomotion and simple control tasks, leaving open how well the principle transfers to the fine-grained wrist-pose prediction setting.

Two further bodies of work establish that offline rankings can transfer to physical outcomes when the evaluation domain is well-calibrated.

SimplerEnv establishes the correspondence between simulation-based evaluation and physical robot performance for visuomotor policies [42]. The study evaluates multiple manipulation tasks and policy types in simulation and then physically, using a WidowX robot under matched visual and control conditions. Rankings produced from simulation-only evaluation closely track rankings from physical trials, with correlations holding across both policy families and task types. The key condition for transfer is domain calibration: visual appearance, camera viewpoint, and control interface must match between the simulation and the physical system. When these are well-matched, simulation rankings provide a reliable ordering of policies without requiring physical deployment.

WorldEval extends the validation to the world-model setting [43]. Rather than evaluating policies in a manually constructed simulator, WorldEval scores them using a learned world model that predicts future states from offline trajectory data. The world-model-based rankings correlate strongly with real-world task success: policies that score well in world-model evaluation also tend to succeed when deployed physically.

Both results carry a shared qualification: rankings hold under well-calibrated conditions, where the evaluation domain is consistent with the training distribution and the deployment context. When this alignment breaks down, offline rankings can degrade. The design implications of this constraint are discussed in Chapter 3.

Summary. The literature surveyed in this chapter establishes three supporting findings. First, world model rankings are informative evaluators both before and during execution, as demonstrated by WorldGym and CLOVER [27, 28]. Second, latent predictive objectives produce more efficient and better-calibrated representations than pixel-level reconstruction, as demonstrated by the JEPA family [21, 31]. Third, offline evaluation rankings can carry predictive validity for physical outcomes when the evaluation domain is well-calibrated, as demonstrated by SimplerEnv and WorldEval [42, 43].

These findings leave two questions unanswered, which this dissertation addresses directly.

RQ1. Does augmenting a fixed imitation-learning policy with a latent world model evaluator improve candidate selection quality in an offline dexterous manipulation benchmark, and does any improvement hold across policy families?

The offline reinforcement learning literature establishes the need for principled candidate selection but benchmarks only locomotion and simple control tasks [38–41]. No controlled study isolates the world model evaluator's contribution on a fixed dexterous manipulation policy.

RQ2. Does the world model's representational strategy — pretrained video backbone, task-data-trained from scratch, or out-of-domain modality — produce meaningfully different selection guidance when applied to the same policy on the same evaluation data?

The interaction between world model architecture and policy performance has not been examined in this setting. Answering RQ2 requires a benchmark that holds the policy and evaluation surface fixed while varying only the world model family. The methodology for constructing and running such a benchmark is described in Chapter 3.

3 Methodology

This chapter describes the experimental design used to evaluate whether latent world model augmentation improves candidate selection quality when paired with imitation-learning policies, and whether any observed effect generalises across policy families and world model architectures. Five design decisions structure the evaluation: the choice of dataset and action space (Section 3.1); a three-branch framework that isolates the contribution of each selection mechanism (Section 3.2); the selection of two policy families that span distinct generation mechanisms (Section 3.3); the selection of five world model families that cover a range of pretraining regimes and input modalities (Section 3.4); and a fair experimental protocol that normalises training exposure across all trained components (Section 3.5). Evaluation metrics and scope limitations are addressed in Sections 3.6 and 3.7.

3.1 Dataset and Experimental Setup

3.1.1 EgoDex: source and selection rationale

The benchmark is built on EgoDex [44], a large-scale egocentric dataset of bimanual dexterous manipulation collected using Meta Quest Pro head-mounted cameras. It spans more than 100 task categories across its full release, including folding, slotting, pouring, and kneading, covering a wide range of fine-grained hand motions. The full release comprises five training parts and supplementary additional-data splits; this study uses the first three training parts, which together comprise more than 194,000 demonstration episodes and approximately 1.3 TB of paired video and annotation data. Each episode provides synchronised RGB video and full six-degree-of-freedom SE(3) wrist annotations for both hands at approximately 30 frames per second.

The choice of dexterous manipulation as the evaluation domain is not incidental. This dissertation forms part of the University of Malta's ongoing M.AI.ESTRO research project [45], which investigates the use of AI for scene and task prediction on dexterous manipulation tasks to support industrial transfer learning and human skill digitisation. The research domain was therefore set by this institutional context: the methodology operates on human hand demonstrations rather than robotic end-effector data because the target application is the transfer of tacit human dexterous skill. An evaluation framework built around robot arm trajectories would not address that objective. The introduction situates this motivation more broadly; the present chapter focuses on the methodological consequences of the domain choice.

Within the dexterous manipulation setting, EgoDex was selected on three properties. First, the benchmark requires a dexterous bimanual hand action space: fine-grained articulation across an eighteen-dimensional bimanual wrist pose representation drives the manipulation outcome, and large-scale corpora built around rigid gripper or arm contact do not provide this. Second, the egocentric viewpoint aligns with the intended downstream deployment setting of a wearable assistive device. Third, the dataset's scale and task diversity are sufficient to support simultaneous training of five world model families and two policy architectures on declared non-overlapping splits while retaining a held-out test set of meaningful size.

EgoDex carries two acknowledged limitations. The demonstrations are produced by human subjects rather than a robotic system, which introduces a kinematic domain gap between the training distribution and any robotic deployment. No contact or force signals are present; the action representation is purely kinematic SE(3). Both are accepted as known constraints rather than experimental variables and are discussed further in Section 3.7.

3.1.2 Windowing protocol

The dataset is stored as paired `.mp4` and `.hdf5` episode files. A deterministic index stage discovers all valid paired episodes, infers their lengths from HDF5 metadata, and writes canonical observation and prediction windows for each split. This shared index is the single source of truth for all downstream modules: policy training, world model extraction, and evaluation all consume the same window file for their respective split, ensuring that output differences across the experimental branches can be attributed to branch design rather than to differences in the data presented to each component. The benchmark uses a fixed-stride windowing scheme as the canonical contract; a supplementary state-change-targeted scheme was developed during early exploration and used exclusively for qualitative inspection.

Strategy 1: Fixed-stride sampling. Each window spans 16 observation frames followed by 16 prediction frames, with consecutive window starts separated by a stride of 128 frames. At 30 frames per second this places starts approximately 4.3 seconds apart, corresponding to an observation context and prediction horizon of approximately 0.53 seconds each. This contract is encoded as `obs16_pred16_s128` and forms the foundation for all policy training and world model training (see Figure 3.1).

Strategy 2: State-change-targeted sampling. The fixed-stride scheme samples the episode timeline uniformly, which causes it to under-sample high-motion events: long near-static segments contribute windows at the same rate as active manipulation periods. As detailed in Chapter 4, this limitation motivated a revised strategy adopted for a subsidiary perceptual inspection task and qualitative development review. Rather

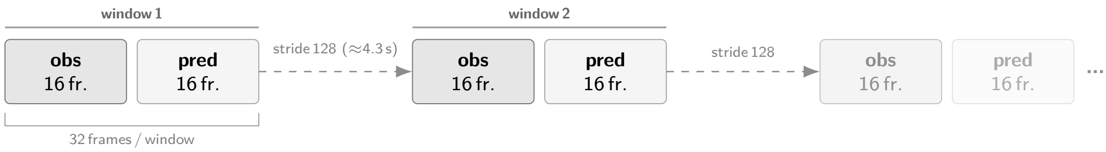

Figure 3.1 Fixed-stride windowing scheme (`obs16_pred16_s128`) used as the canonical training and evaluation index. Author-generated schematic.

than placing windows at fixed intervals, this approach computes smoothed hand-state change scores from the wrist transform and confidence sequences, selects candidate windows near high-change peaks, and applies episode-first task-stratified sampling with a fixed random seed (see Figure 3.2). A relative-position fallback retains slower episodes that would otherwise be excluded, yielding one frozen window per episode stratified across all represented task categories.

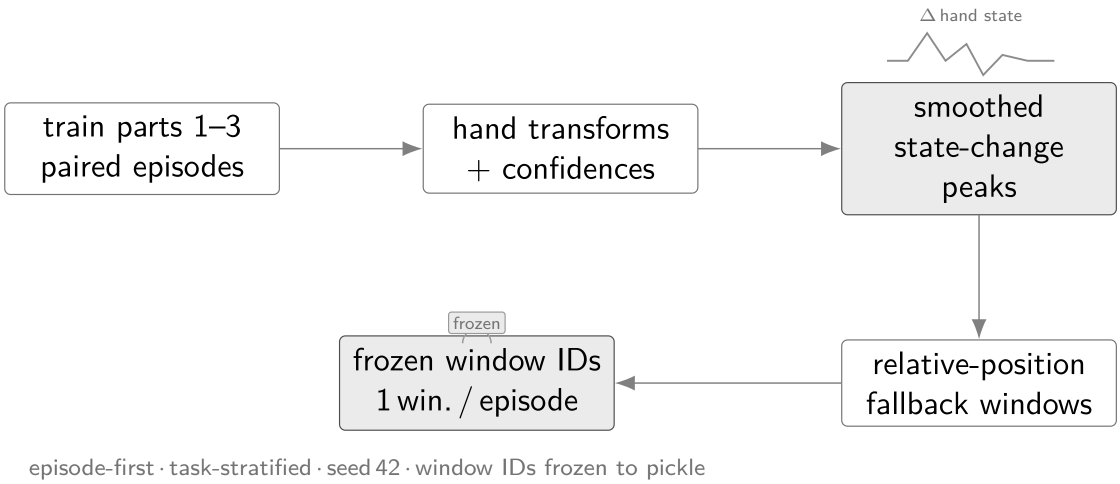

Figure 3.2 State-change-targeted sampling pipeline yielding one frozen window per episode. Author-generated schematic.

The canonical training and evaluation index uses the fixed-stride scheme across all experimental branches. The state-change-targeted subset was used exclusively for a subsidiary perceptual inspection task and qualitative development review; it does not form part of the primary benchmark evaluation.

3.1.3 Training splits and test set

Three cumulative training splits were assembled from the first three parts of the EgoDex release, each a strict superset of the previous; Table 3.1 summarises their composition. EgoDex ships five training parts plus supplementary additional-data splits; parts four and five and all additional-data splits are excluded from this study to bound the experimental scope. The held-out test split is the official EgoDex test set and is never used during training at any stage by any model or policy.

Table 3.1 EgoDex splits under the `obs16_pred16_s128` windowing contract. Episode and window counts are derived from the canonical index.

Split	Tasks	Episodes	Windows	Role
`train_part1`	26	46,091	140,239	Stage 1 training
`train_part1_2`	39	140,654	291,369	Stage 1+2 training
`train_part1_2_3`	63	194,333	430,962	**Canonical training**
`test`	111	3,229	7,607	**Frozen evaluation**

The episode counts reflect the uneven collection density across parts: the 13 task categories added in part two average approximately 7,300 episodes each, roughly four times the average of the 26 part-one tasks, because part two focuses on high-frequency foundational primitives such as pick-and-place and folding that were collected at greater volume.

The canonical results in Chapter **??** use `train_part1_2_3` across all trained components. The test split is the official EgoDex held-out set spanning 111 task categories, which includes all 63 categories present in `train_part1_2_3` as well as 48 categories drawn from the excluded parts; evaluating on this broader set provides a measure of out-of-distribution generalisation. Its window index is frozen; no post-hoc filtering or re-sampling is applied. The incremental training curriculum across the three stages is described in Chapter 4.

3.2 Three-Branch Experimental Framework

The central question of this study is whether a latent world model improves candidate selection quality when paired with an imitation-learning policy, and whether that effect generalises across policy families. Answering both questions cleanly requires holding every other experimental factor constant. The benchmark therefore organises evaluation around three branches that share the same policy weights, the same evaluation windows, and the same candidate pool across B2 and B3; only the selection rule applied to the candidate pool differs (Figure 3.3). Any measured performance difference between branches is directly attributable to the selection mechanism rather than to training data, architecture, or window coverage.

Two policy families are evaluated within this framework: ACT, whose CVAE-Transformer decoder generates five candidate action chunks in a single forward pass, and Diffusion Policy (DP), which generates five candidates via five independent Denoising Diffusion Implicit Models (DDIM) denoising chains on the same observation. Both families are given equal standing; the candidate pool contract is identical for each, and all three branches apply the same selection rules to both.

Branch 1 (B1): Policy-only baseline. The policy generates its full candidate pool and selects the candidate it scores highest by its own internal preference. No world model is queried at runtime. B1 establishes the trajectory prediction quality the policy achieves acting on its own judgement alone and serves as the baseline against which both world-model branches are measured.

Branch 2 (B2): Hybrid with world-model veto. The policy generates the same five candidate action chunks. The world model scores all five by predicting forward in its latent space from the current observation and measuring the fidelity of each prediction. If the world model's preferred candidate differs from the policy's preferred candidate and its score exceeds the policy's choice by a declared margin threshold, the world model overrides the selection; otherwise the policy's preferred candidate stands. The threshold is a fixed scalar applied uniformly across all world model families; its value and the empirical procedure used to select it are reported in Chapter 4. This conservative design confines world-model authority to windows where it has sufficient relative confidence. The mechanism is analogous in spirit to model-based offline reinforcement learning approaches such as MOReL [39] and MOPO [38], which use a learned model's uncertainty to constrain policy decisions to regions where the model is reliable.

Branch 3 (B3): World-model-driven selection. The margin threshold is removed entirely. The world model selects the candidate with the highest score unconditionally. Critically, B3 does not re-run the policy: it operates over the frozen candidate pool produced during the B2 pass for each window. The policy contributes its proposals at B2 time; B3 reuses them without generating any new candidates. This design ensures that comparing B2 against B3 isolates the effect of removing the threshold gate on identical inputs, with no other variable changing.

The isolation property holds at every level of the evaluation. All 7,607 test windows are fixed and shared across all branches and all five world model families. The five-candidate pool per window is generated once and reused across B2 and B3. World model weights are fixed at the end of training and not updated during evaluation. The only degree of freedom is the selection rule applied to the pool. Differences in measured trajectory prediction quality between branches therefore reflect the causal effect of world model involvement in selection.

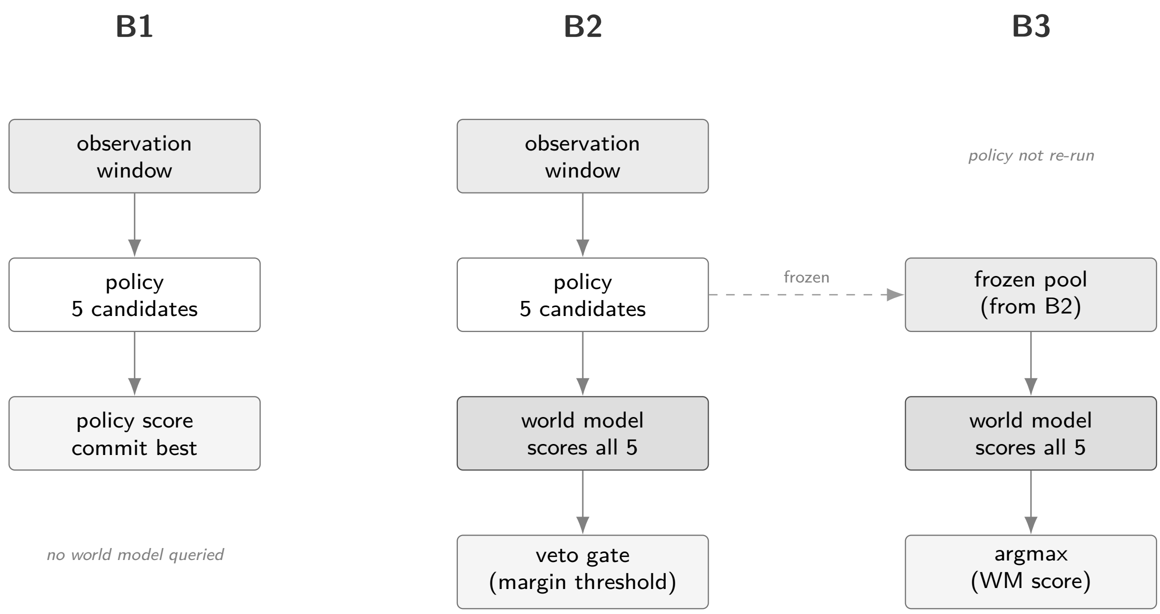

Figure 3.3 Three-branch design space. B1 uses the policy's own preference alone; B2 adds a world-model veto constrained by a margin threshold; B3 gives the world model full selection authority over the frozen candidate pool produced at B2 time. The policy is not re-run in B3.

3.3 Policy Variants

The benchmark evaluates two imitation-learning policy families. Their architectures and training objectives are reviewed in Chapter 2; the properties relevant to this study are how each generates its candidate pool and what that implies for world-model scoring.

- **Action Chunking Transformer** (Section 2.1). ACT samples five latent vectors from its CVAE prior and decodes them in a single forward pass, producing five candidate action chunks simultaneously. The candidate pool therefore requires exactly one network evaluation. Each candidate is a 16-step bimanual SE(3) wrist-transform sequence derived from the same observation-conditioned latent distribution; diversity is bounded by the width of that learned prior.
- **Diffusion Policy** (Section 2.2). DP generates each candidate through an independent reverse-diffusion chain using a DDIM [46] schedule of ten steps. Producing five candidates requires five separate chains and approximately fifty network evaluations per window, roughly an order of magnitude more than ACT. The Diffusion Transformer (DiT) backbone is used. Candidates are structurally independent because each starts from a fresh noise sample; diversity arises from stochastic initialisation rather than from a shared latent manifold.

Evaluating both families under the same three-branch protocol allows the study to ask whether world-model augmentation improves candidate selection in general or only under a particular generation mechanism. A result that holds across both constitutes stronger evidence for the generality of the approach.

3.4 World Model Families

Five world model families are evaluated as candidate selectors within the three-branch framework. Their full architectures and pretraining objectives are reviewed in Chapter 2; this section describes how each family is positioned in the experimental design and what methodological role it plays. The five families span a deliberate range: from a large pretrained video backbone through a lightweight from-scratch latent model to an out-of-domain negative control, providing multiple comparison axes beyond a single-model result.

In all cases, the world model's task at evaluation time is the same: given the current observation and each of the five candidate action chunks produced by the policy, score the candidates by predicting forward in the model's latent space and returning a scalar quality measure. The policy weights and evaluation windows are fixed; the only variable is which world model supplies the scores.

V-JEPA2-AC — Pretrained video backbone. The architecture is reviewed in Section 2.3.2. A frozen V-JEPA2 [31] Vision Transformer (ViT)-Large backbone provides a 1024-dimensional frame embedding pretrained on large-scale video via the JEPA objective. A lightweight action-conditioned transition head, trained on benchmark data, maps a current embedding and an action chunk to a predicted future embedding; the L2 distance between prediction and observed future embedding serves as the candidate score. This family represents the upper end of the representational hierarchy: rich, general-purpose video features acquired at scale before any manipulation-specific training.

LeWM — From-scratch latent world model. The architecture is reviewed in Section 2.3.2. LeWM [34] is trained entirely from raw EgoDex pixels with no pretrained weights. A ViT-Tiny encoder and decoder are jointly optimised via a next-embedding prediction loss and a Gaussian regularisation term, without exponential moving average stabilisation or reconstruction of pixel values. This family represents the opposite end of the pretraining spectrum: all representations are learned from the benchmark training data alone, making it a provenance-clean baseline with no external data dependency.

DexWM — Domain-specific architecture. The architecture is reviewed in Section 2.3.2. Dexterous Manipulation World Model (DexWM) [33] pairs a frozen DINOv2 image encoder with a CDiT predictor and incorporates a hand-consistency loss that explicitly supervises fingertip and wrist heatmap predictions. It is included to assess whether a model built with explicit hand-geometric supervision outperforms general video predictors on this task. A constraint on the training setup applies: as described in Section 3.7, publicly available model weights could not be sourced and the model was therefore trained from scratch on the benchmark's declared training splits, adopting the same provenance stance as LeWM: no external pretraining and no data outside the declared splits.

V-JEPA2 (no action conditioning) — Ablation control. This family uses the same frozen V-JEPA2 backbone as V-JEPA2-AC but omits the action-conditioned transition head. The proposed action chunk is not passed to the model; the future embedding is predicted from visual context alone via a learned visual-only prediction head (implementation details in Chapter 4), and the candidate score reflects visual plausibility rather than action-conditional consistency. The purpose is to isolate the contribution of action conditioning: if V-JEPA2-AC outperforms this variant, the advantage is attributable to the model's ability to reason about action consequences rather than to the representational power of the backbone alone.

PointWorld — Out-of-domain negative control. The architecture is reviewed in Section 2.3.2. PointWorld [36] is a 3D point-flow world model trained on robotic manipulation data in a fundamentally different modality and domain from EgoDex. It operates on 3D point clouds rather than egocentric RGB video, and its training distribution does not include human hand demonstrations. Its inclusion as a negative control tests the specificity of the world-model benefit: a well-calibrated result should show PointWorld performing significantly worse than the in-domain families, confirming that the observed gains reflect genuine latent video prediction rather than an artefact of the evaluation protocol.

Table 3.2 World model families and their methodological roles in the benchmark.

Family	Pretraining	Modality	Role
V-JEPA2-AC	Large-scale video	RGB video	Primary pretrained baseline
LeWM	None (scratch)	RGB video	Provenance-clean from-scratch
DexWM	None (scratch)*	RGB video	Domain-specific (hand-geometry loss)
V-JEPA2 (no AC)	Large-scale video	RGB video	Action-conditioning ablation
PointWorld	Robot point clouds	3D points	Out-of-domain negative control

* The published DexWM architecture targets pretraining on EgoDex and DROID; publicly available weights could not be sourced and the model used here was trained from scratch (Section 3.7).

The modality column reports each family's primary input modality. The four RGB video families do not predict or reconstruct pixels: as described in Section 2.3.2 of the literature review, all JEPA-family and encoder-based architectures translate the

RGB input into a learned latent representation and perform all prediction and scoring operations in that latent space.

Together, the five families allow comparisons along three independent axes: pretraining scale (V-JEPA2-AC vs LeWM), action conditioning (V-JEPA2-AC vs V-JEPA2 no-AC), and modality (all video-based families vs PointWorld). No single family can be claimed responsible for a result that generalises across all three axes.

3.5 Fair Experimental Protocol

Comparing the five world model families and two policy architectures against each other requires holding every non-design variable constant. Two aspects of fairness are addressed: training exposure and evaluation contract.

3.5.1 Training exposure budget

Different model families have different computational costs per training step and different sensitivities to data volume. Without a principled exposure budget, a comparison could conflate the effect of architecture with the effect of receiving more or fewer training samples. To determine an appropriate shared budget, a convergence study was conducted prior to full training: each model family was trained independently at six data scales (1k, 5k, 10k, 25k, 50k, and 100k effective items) on a fixed subset of the canonical training data, and the resulting loss curves were examined for plateau behaviour. The study and its results are described in detail in Chapter 4; the finding relevant to the experimental design is summarised here.

The effective training exposure E is defined as:

$$E = S \times B \times G \times A \tag{3.1}$$

where S is the number of optimisation steps, B is the per-device batch size, G is the number of gradient-accumulation steps, and A is the number of accelerator devices. This formulation counts the number of training items consumed regardless of how steps are distributed across hardware configurations, allowing models with different batch sizes and GPU counts to be normalised to a common scale.

The convergence study, described in Chapter 4, established a universal budget of $E = 100{,}000$ effective items. This figure is applied uniformly across all model families and all three training splits, ensuring that any measured performance difference reflects architecture and data rather than exposure volume.

3.5.2 Evaluation contract

All models receive the same RGB input at evaluation time: the observation frames from the canonical test window, without additional depth, point cloud, or semantic metadata. This ensures that any performance difference between world model families reflects their internal representation and prediction quality rather than access to a richer input signal. PointWorld, which natively operates on 3D point clouds, is evaluated after projecting test observations into point cloud representations via a depth-estimation encoding step; the specific model used and any associated systematic artefacts are documented in Chapter 4.

The evaluation windows are fixed at indexing time and shared across all branches and all world model families. No window is selected, filtered, or re-sampled after training. Policy weights and world model weights are frozen before evaluation begins and are not updated. The only variable at evaluation time is the selection rule applied to the candidate pool, isolating the contribution of each branch design and each world model family from all other sources of variance.

3.6 Evaluation Metrics

3.6.1 Offline trajectory prediction setting

EgoDex does not define an official offline evaluation metric. The original benchmark measures policy effectiveness via online task success rate on a physical robot deployment [44], which is outside the scope of this study. The evaluation setting here differs fundamentally: rather than measuring task execution, the study evaluates offline visuomotor trajectory prediction, measuring how closely each configuration's predicted wrist pose sequences align with ground-truth demonstrated trajectories in the held-out test set. Results should therefore be interpreted as trajectory prediction quality under each branch and world model configuration, not as claims about downstream task success. The relationship between offline trajectory prediction quality and online task performance is addressed in Section 2.4 of the literature review and in Section 3.7.

3.6.2 Primary metrics

The action representation throughout this study is a sequence of SE(3) wrist transforms for both hands (see Section 3.1). Evaluation metrics are defined over this representation directly, decomposed into translation and rotation components.

Translation L2 error. For each predicted action chunk, the Euclidean distance between the predicted and ground-truth wrist positions is computed at each of the 16 prediction steps and averaged across the chunk and across both hands. This yields a mean per-step translation error in metres, providing an interpretable measure of spatial accuracy over the prediction horizon.

Rotation error. The geodesic distance between the predicted and ground-truth wrist orientations is computed at each step, expressed in degrees. As with translation, values are averaged across the chunk and across both hands. Geodesic distance on SO(3) is used in preference to Euler-angle difference, which is sensitive to gimbal singularities and axis ordering.

The following supplementary metrics are computed alongside the two primaries and reported for completeness. All are expressed as pairwise win rates against the B1 baseline unless stated otherwise.

- **Rotation win rate.** Fraction of windows where the selected candidate achieves lower rotation error than B1; the rotational counterpart to the translation win rate.
- **Endpoint accuracy win rate.** Win rate computed over the mean translation error of the final four prediction steps only, capturing whether selection quality holds through to the end of the prediction horizon.
- **Velocity win rate.** Fraction of windows where the selected candidate's step-to-step velocity profile (first-difference of wrist positions) is closer to the ground-truth velocity profile than B1's selection.
- **Acceleration win rate.** Same as velocity win rate, computed on the second-difference (acceleration) of the wrist position sequence.
- **Straightness win rate.** Fraction of windows where the selected candidate's trajectory is geometrically closer to the ground-truth path shape, penalising unnecessary curvature.
- **Oversmoothing win rate.** Fraction of windows where the selected candidate avoids excessive temporal smoothing relative to the demonstrated trajectory; complements straightness by penalising insufficient motion variation.
- **Nonlinear acceptance rate (B2 only).** Fraction of test windows in which the B2 veto gate fires, i.e. the world model's preferred candidate differs from the policy's and its margin exceeds the threshold. This is a diagnostic for B2 rather than a quality metric; it characterises how often the world model actively intervenes.

3.6.3 Candidate selection comparison

Comparing headline metric values directly across world model families is complicated by the fact that the five families produce scores on different numerical scales, and that absolute metric differences may reflect policy rather than world model quality. To support clean cross-family comparison, candidate selection is also evaluated via a pairwise win rate: for each test window, the candidate selected by a given branch or world model family is compared against the candidate selected by B1 (policy-only), and a win is recorded if the selected candidate achieves lower translation L2 on the ground-truth trajectory; translation L2 is used as the win criterion because it provides the most spatially direct and interpretable measure of selection quality across the 16-step prediction horizon. Win rates are averaged across all test windows (Table 3.1) and reported as percentages. This measure is scale-free and directly answers the question of whether world-model involvement produces a better selection decision than the policy alone, independent of the absolute metric level. The offline evaluation literature supports pairwise comparison as a robust ranking method [43].

3.7 Evaluation Scope and Limitations

This section collects the scope constraints and known limitations of the experimental design. Several have been introduced at relevant points in earlier sections; they are gathered here to give a complete account.

Offline evaluation only. The benchmark is entirely offline: no physical robot is used, and no task completion signal is available. All metrics are computed against ground-truth demonstrated trajectories in the held-out test set. The consequence is that a configuration may achieve strong trajectory prediction scores while still failing at the physical task, and vice versa. The literature reviewed in Section 2.4 establishes that offline rankings can carry predictive validity for physical outcomes when the evaluation domain is well-calibrated to the deployment setting, but this transfer is not guaranteed. The results of this study should be read as evidence about candidate selection quality in the offline trajectory prediction setting, not as claims about downstream robot task success.

Kinematic action space. The action representation is limited to SE(3) wrist pose transforms for both hands: nine dimensions per wrist (a six-dimensional rotation encoding and three translational components), eighteen dimensions in total across the bimanual system. Finger-level articulation is not represented. The evaluation therefore measures bimanual wrist trajectory quality; conclusions do not extend to finger-level dexterous control or to tasks where outcome depends on fine-grained in-hand manipulation.

No contact or force signals. EgoDex provides RGB video and wrist pose annotations; no contact, force, or haptic signals are present. The action representation is purely kinematic. Tasks for which contact dynamics are the determining factor cannot be evaluated in this setting.

Kinematic domain gap. Demonstrations are produced by human subjects, not a robotic system. The wrist kinematics of a human arm differ from those of a robot manipulator in range of motion, joint compliance, and embodiment. Any downstream robotic deployment would require bridging this domain gap, which is not addressed by the offline benchmark.

Training budget as a compute-constrained lower bound. As noted in Section 3.5.1, the universal training budget of 100k effective items was determined empirically but is not a saturation point for all model families. Some families were still improving at this scale. Performance differences between families may partly reflect differing distances from their respective saturation budgets rather than intrinsic architectural differences.

DexWM trained from scratch. The published DexWM architecture [33] does not provide publicly available model weights: the associated repository contains only trajectory data. As shown in Chapter 4, the pretrained checkpoint could not be sourced and DexWM was therefore trained from scratch on the benchmark's declared training splits, following the same procedure as LeWM. This means DexWM results reported here are provenance-clean with respect to the test split, but the trained model does not carry the domain-specific pretraining on EgoDex and DROID that the published architecture was designed to exploit. Results for this family should be interpreted accordingly.

4 Implementation

This chapter describes the concrete implementation decisions made to realise the experimental design specified in Chapter 3. Six concerns structure the account: the repository layout and execution model (Section 4.1); the data pipeline and windowing infrastructure (Section 4.2); how each policy and world model family was trained (Sections 4.3 and 4.4); the convergence study that determined the universal training budget (Section 4.5); the calibration procedure for the B2 margin threshold (Section 4.6); and the guard evaluation infrastructure together with the fairness safeguards built into it (Sections 4.7 and 4.8). Implementation challenges that required significant diagnosis and correction are documented in Section 4.9; auxiliary modules excluded from the primary benchmark are described in Section 4.10.

4.1 Repository and execution model

The research stack is organised as a single Python package under `src/`, with subsystem entry points exposed under `scripts/`. The canonical shell surface consists of `policy.sh`, `jepa_wm.sh`, `lewm.sh`, `docs.sh`, and the higher-level orchestration wrapper `run_stage.sh`. Docker is the canonical runtime surface for all stages: datasets, caches, and run outputs are kept outside the repository checkout so that code, data, and generated artefacts remain separated. All reported results are outputs of this staged execution model, tied to frozen run artefacts rather than to manually reconstructed notebook states.

The run output directory follows a split-first, then model layout. Each training job writes its artefacts under `fair_training/<split>/<model>/`, which includes a `result.json` recording the completion status, wall time, and final metric. This layout makes it straightforward to audit which combinations have completed and to compare results across splits for the data-scaling analysis.

4.2 Data pipeline and windowing

4.2.1 Index construction

The index stage is the foundation of the entire stack. It discovers valid paired EgoDex episodes, requiring both a `.mp4` video file and a `.hdf5` pose annotation file, infers episode lengths from the Hierarchical Data Format 5 (HDF5) metadata, and writes canonical observation and prediction windows. For an episode of length T, a window at position t_0 is valid when both the observation slice $[t_0,\ t_0 + 16)$ and the prediction slice $[t_0 + 16,\ t_0 + 32)$ fall within the episode boundaries. The canonical contract used throughout this study is `obs_len=16`, `pred_len=16`, `stride=128`, producing windows of 16 observed frames followed by 16 frames to predict, sampled every 128 frames along each episode. This contract is encoded as `obs16_pred16_s128` and is described in Section 3.1.2 of the methodology.

The shared window definition enforces the isolation property central to the experimental design. Policy training, world model extraction, and guard evaluation all consume the same window file for their respective split. Any performance difference between branches or world model families is therefore attributable to branch design or architecture, not to differences in the data presented to each component.

The index stage writes a resumable checkpoint during construction, allowing an interrupted build to continue from the last completed episode rather than starting from scratch. For the largest split (`train_part1_2_3`, approximately 1.3 TB and 194,333 episodes) this resumability was practically necessary because the indexing job ran over several hours.

4.2.2 Incremental training curriculum

Three cumulative training splits were assembled from the first three parts of the EgoDex release. `train_part1` covers 26 task categories and 140,239 windows; `train_part1_2` extends this to 39 task categories and 291,369 windows; `train_part1_2_3` covers 63 task categories and 430,962 windows. Each split is a strict superset of the previous: the larger splits were assembled using relative symbolic links rather than copying data, allowing incremental construction without duplicating approximately 1 TB of earlier data on disk.

This curriculum allows the data-scaling axis of the benchmark to be evaluated: all five world model families and both policy architectures are trained independently on each of the three splits at the universal budget of 100,000 effective items (Section 4.5), enabling a direct measure of how additional training data diversity affects prediction quality independent of training exposure volume.

4.3 Policy training

Both policy families share the same observation encoder: a ResNet-18 visual backbone processes the single observation RGB frame (96×96 pixels) and produces a feature vector that is concatenated with the bimanual wrist state before being passed to the policy-specific decoder. Image resizing to 96×96 is applied at loading time and is consistent across all branches. Wrist pose sequences are normalised to zero mean and unit variance using statistics computed from the training split; these normalisation statistics are frozen after training and consumed by the world model scoring stages to ensure consistency.

Action Chunking Transformer. The ACT architecture is reviewed in Section 2.1. At training time, the CVAE encoder compresses a ground-truth action chunk and the current observation into a style variable z, which the transformer decoder then uses to reconstruct the chunk. The encoder is discarded at evaluation time; five candidate chunks are produced by sampling z independently from the Gaussian prior five times and decoding each under the same observation.

ACT training used single-node distributed data-parallel execution via `torchrun` across two A100 GPUs, with a per-device batch size of 32 (global batch 64). At the universal budget of 100,000 effective items, this corresponds to 1,562 optimisation steps. Training on `train_part1_2_3` completed in approximately 14 GPU-minutes per run (around 820 wall-clock seconds), making ACT the lightest training task in the benchmark. Each run writes a `best.pt` checkpoint selected by lowest validation loss, a `last.pt` checkpoint, per-epoch metrics in `metrics.jsonl`, and a frozen `manifest.json` recording the configuration and window file consumed.

Diffusion Policy. The Diffusion Policy implementation follows the transformer variant (DP-T) described in Section 2.2. The same ResNet-18 observation encoder produces a conditioning vector `obs_cond` that is passed to a DiT denoising backbone via cross-attention. At evaluation time, five independent DDIM denoising chains are run on the same observation to produce five candidate action chunks; each chain starts from independently sampled Gaussian noise and converges to a distinct trajectory under the shared conditioning signal.

Diffusion Policy training ran on a single A100 GPU with a per-device batch size of 32. At the universal budget of 100,000 effective items this corresponds to 3,125 optimisation steps. Training on `train_part1_2_3` completed in approximately 2.2 GPU-hours per run, making Diffusion Policy approximately ten times more expensive per run than ACT despite the lower parameter count, because each training step requires a full forward pass through the noise prediction network over the DDPM noise schedule. The same checkpoint, metrics, and manifest artefacts as ACT are written at the end of each run.

4.4 World model training

4.4.1 V-JEPA2-AC: extraction and transition head

The V-JEPA2-AC family separates training into two phases. In Phase 1, the frozen V-JEPA2 ViT-Large encoder processes all indexed windows in a single batched inference pass. The encoder takes 16-frame observation sequences and produces a latent embedding vector for each window; these embeddings are written to `embeddings.npy` on disk, one row per window, together with a companion `window_ids.txt` that records the exact identifier (split, task, episode, t_0) of each row. Phase 1 is not a training job: there is no backward pass, no optimiser, and no epoch loop. It is an offline feature extraction that runs once per split and stores the results permanently.

The motivation for this caching strategy is straightforward. The 300-million-parameter ViT-Large processes 16-frame sequences; running it over every training window at each training epoch would be computationally prohibitive, and is unnecessary because the backbone is frozen. For the `train_part1_2_3` split, Phase 1 extracted embeddings for 430,962 windows using sharded multi-GPU execution across four A100 GPUs. The window list was partitioned across devices, and a merge step concatenated the per-shard files after all shards completed, verified by comparing the window count against the canonical window list. A critical bug in an early extraction run applied a 2,000-window default cap, producing only approximately 1,000 training pairs rather than the expected 94,000 and effectively rendering the first V-JEPA2-AC training runs invalid; this is documented further in Section 4.9.

In Phase 2, the action-conditioned transition head is trained on the stored embeddings. The head is a small multi-layer perceptron that learns to predict the next embedding z_{t+1} from the current embedding z_t and an action conditioning vector derived from the bimanual wrist state delta. The `action_repr=policy_state_delta` conditioning aligns the action representation used in transition-head training with the same bimanual state representation consumed by the policy, ensuring that the world model can score candidate chunks consistently. Phase 2 training ran on four A100 GPUs and completed in under ten seconds at the 100,000-item budget, because all expensive backbone inference is fully amortised in Phase 1. The trained head is exported through an `act_adapter.json` contract loaded by the guard stage at runtime.

4.4.2 LeWM: end-to-end training

LeWM is reviewed in Section 2.3.2. Training jointly optimises the ViT-Tiny encoder, the autoregressive predictor, the action embedder, and the projector from random initialisation, with no frozen backbone and no external pretraining. The training objective combines a next-embedding prediction loss $\mathcal{L}_{\text{pred}}$ with the SIGReg regulariser that enforces a Gaussian prior on the latent distribution [34]:

$$\mathcal{L}_{\text{LeWM}} = \mathcal{L}_{\text{pred}} + \lambda\,\text{SIGReg}(Z) \tag{4.1}$$

EgoDex episodes are first exported into contiguous HDF5 sequences that the

LeWM training loop can consume efficiently without reopening individual episode files at each step. Each exported sequence contains an array of pixel frames, the per-step bimanual wrist state, the per-step action (defined as the state delta between consecutive steps), and per-episode length and byte-offset metadata. The minimum required frame cap must be computed from the actual benchmark window lists: the test split requires coverage up to frame 5,152 and `train_part1_2` up to frame 5,280, so the export contract adopts `max_frames_per_episode=5280` throughout. An earlier cap of 256 frames caused the initial LeWM evaluation to cover only 160 of 7,607 intended test windows (2.1 per cent coverage); the full resolution of this defect is documented in Section 4.9.

LeWM training ran on four A100 GPUs with a per-device batch size of 4 (global batch 16, with four gradient-accumulation steps). At the universal budget of 100,000 effective items this corresponds to 6,250 optimisation steps, making LeWM the most step-intensive of the primary models. Wall-clock training time on `train_part1_2_3` was approximately 47 hours per run.

4.4.3 DexWM: from-scratch training

DexWM is reviewed in Section 2.3.2. Its pipeline pairs a frozen DINOv2 image encoder with a CDiT predictor and incorporates a hand-consistency loss that supervises fingertip and wrist heatmap predictions.

Checkpoint sourcing investigation

The published DexWM architecture [33] does not provide publicly available model weights. The associated HuggingFace repository (`facebook/dexwm`) contains only trajectory data in `.hdf5` format; no `model.pt`, `pytorch_model.bin`, or equivalent weight file is present. The GitHub repository README states that pretraining requires 32 nodes of 8 GPUs each for several days on the EgoDex and DROID datasets, and does not link to a downloadable checkpoint. Automated download attempts confirmed a 404 response for all candidate weight paths; no mirror or third-party release was located.

The codebase initially contained a silent fallback that caught the download failure and substituted an identity stub (`_CDiTStub`), which returned zero scores for all candidates without any user-visible error. This stub was replaced with an explicit hard-failure that surfaces the sourcing failure with guidance, preventing any experiment from silently using zero-score predictions.

Because no external checkpoint could be sourced, DexWM was trained from scratch on the benchmark's declared training splits, following the same provenance stance as LeWM: no external pretraining and no data outside the declared splits. Training used the DINOv2 ViT-Large encoder (1,024-dimensional patch tokens) as a frozen backbone, with the CDiT predictor and hand-consistency head trained from random initialisation. An early run misconfigured the DINOv2 variant as ViT-Base (768-dimensional), causing all guard-stage inference to fail; the fix confirmed the correct 1,024-dimensional configuration and is documented in Section 4.9.

DexWM training ran on four A100 GPUs with a per-device batch size of 4 (global batch 16). At the universal budget of 100,000 effective items this corresponds to 6,250 optimisation steps. Wall-clock training time on `train_part1_2_3` was approximately 47 hours per run, comparable to LeWM but substantially higher in GPU memory footprint due to the patch-level spatial representations produced by the DINOv2 encoder.

4.4.4 V-JEPA2 (no action conditioning): ablation control

The V-JEPA2 no-AC ablation shares the same frozen ViT-Large encoder and pre-extracted embeddings as V-JEPA2-AC and requires no additional training. Its distinguishing feature is the scoring function: rather than conditioning the transition prediction on the proposed action chunk, the scorer passes a zero action vector to the transition model for all candidates. All candidates therefore receive the same predicted next embedding regardless of their action content, and their scores diverge only from support-distance penalties and other action-independent terms. The purpose of this design is to test whether V-JEPA2-AC's selection gains are attributable to action conditioning specifically or merely to the representational quality of the backbone. If the no-AC variant matches V-JEPA2-AC, the backbone representation alone is sufficient; if it underperforms, action conditioning is the causal driver.

The ablation requires no separate training phase beyond the Phase 1 embedding extraction already conducted for V-JEPA2-AC. A dedicated result artefact records the action-blind persistence loss (the mean MSE plus cosine distance between consecutive frame embeddings, evaluated without any action input) as a diagnostic metric confirming that the ablation ran correctly.

4.4.5 PointWorld: out-of-domain proxy

PointWorld [36] is a 3D point-flow world model trained on robot arm manipulation data with a six-to-seven degree-of-freedom joint action space. Running the full PointWorld inference pipeline on EgoDex observations requires lifting RGB frames to 3D point clouds via a monocular depth estimation model and projecting the bimanual hand action to the robot arm action space expected by the model. This out-of-domain application is the mechanism that makes PointWorld a meaningful negative control: the evaluation deliberately places the model in a domain for which it was not designed.

In this study, PointWorld is evaluated through a proxy implementation that captures the domain mismatch analytically rather than through full 3D inference. The proxy reuses the pre-extracted V-JEPA2 frame embeddings and quantifies two sources of domain mismatch: a persistence loss measuring the frame-to-frame embedding distance (equivalent to action-blind latent consistency), and an action mismatch penalty measuring how poorly the bimanual hand action maps to the robot arm action space. The action mapping uses a `HandToArmActionAdapter` that projects the 18-dimensional bimanual wrist representation (six-dimensional rotation encoding plus three translational components per wrist) to a seven-dimensional robot arm representation (six wrist dimensions plus a scalar gripper aperture derived from the mean of the remaining dimensions) via the `wrist_plus_gripper` strategy. The composite proxy loss is:

$$\mathcal{L}_{\text{PointWorld-proxy}} = \mathcal{L}_{\text{persist}} + 0.5 \cdot \text{mismatch} \tag{4.2}$$

where $\mathcal{L}_{\text{persist}}$ is the persistence loss and mismatch is the fractional norm loss of the projection residual. A well-calibrated negative control should show high mismatch scores, confirming that the bimanual hand action space does not translate cleanly to the robot arm space and that PointWorld's representations are not suited to this domain. Like the V-JEPA2 no-AC ablation, the proxy requires no additional training beyond the Phase 1 extraction shared with V-JEPA2-AC.

4.5 Convergence study and universal training budget

As described in Section 3.5.1 of the methodology, a convergence study was conducted prior to full fair training to determine an appropriate shared exposure budget. Each of the five primary model families was trained independently at six data scales (1k, 5k, 10k, 25k, 50k, and 100k effective items) on a fixed stratified subset of `train_part1_2_3`, and the resulting validation losses were examined for plateau behaviour. The two control families (V-JEPA2 no-AC and PointWorld) were included in the sweep but excluded from the budget selection decision.

4.5.1 Loss curves and plateau detection

Table 4.1 reports the validation loss at each scale for all seven families. Because each family uses a different training objective, raw loss values are not comparable across rows; the relevant signal is each family's own trend across scales.

Table 4.1 Validation loss at each training scale for all seven model families. Losses are not comparable across rows (each family uses a different objective). The relevant signal is each family's trend across scales. The universal budget of 100k is highlighted.

Family	**1k**	**5k**	**10k**	**25k**	**50k**	**100k**
ACT	0.134	0.134	0.129	0.133	0.130	**0.129**
Diffusion	10.86	5.22	4.25	3.11	2.57	**2.14**
DexWM	4.31	2.41	2.00	1.90	1.69	**1.36**
LeWM	0.335	0.209	0.165	0.145	0.130	**0.130**
V-JEPA2-AC	1.796	0.375	0.308	0.286	0.257	**0.227**
V-JEPA2 (no AC)	0.460	0.477	0.373	0.352	0.353	0.377
PointWorld	0.655	0.679	0.628	0.629	0.632	0.592

ACT is effectively flat from 1k onwards: a 4 per cent drop over a 100-fold increase in data volume indicates that the model's loss is dominated by task ambiguity at this data scale rather than by the amount of training data. LeWM plateaus between 50k and 100k (a 0.5 per cent improvement), indicating near-saturation. V-JEPA2-AC, Diffusion Policy, and DexWM are all still declining at 100k: Diffusion drops 17 per cent from 50k to 100k, DexWM drops 20 per cent, and V-JEPA2-AC drops 11 per cent. These three families have not reached their saturation budgets within the sweep range.

The two controls behave as expected. V-JEPA2 no-AC shows no consistent downward trend across scales, confirming that action-blind prediction receives no benefit from more data: without action conditioning, additional training pairs cannot improve the model's ability to discriminate between candidate actions. PointWorld is similarly flat across all six scales, consistent with the proxy measuring a structural domain mismatch rather than a learnable relationship.

4.5.2 Universal budget decision

The plateau budget for each primary family is summarised in Table 4.2. The universal budget is set by the maximum plateau budget across primary models: since Diffusion Policy, DexWM, and V-JEPA2-AC are still data-hungry at 100k, the budget is set to 100k as a compute-constrained lower bound. Families that plateau earlier (ACT at 1k, LeWM at 50k) receive the same 100k budget by design: the extra exposure cannot hurt them and ensures all families are compared under the most favourable common condition reachable within the compute envelope.

This budget is a compute-constrained lower bound rather than a true saturation point for all families. The performance gaps between families reported in the results chapter may partly reflect differing distances from their respective saturation budgets rather than intrinsic architectural differences. This limitation is acknowledged in

Table 4.2 Plateau budget per primary model family and reasoning. The universal budget is $E = 100{,}000$ effective items.

Family	**Plateau budget**	**Evidence**
ACT	1k	Loss flat from 1k; 4% drop over 100$\times$ more data
LeWM	50k	50k$\rightarrow$100k improvement of 0.5%; effectively converged
V-JEPA2-AC	$>$100k	11% drop at 50k$\rightarrow$100k; still declining
Diffusion	$>$100k	17% drop at 50k$\rightarrow$100k; still declining steeply
DexWM	$>$100k	20% drop at 50k$\rightarrow$100k; still declining
Universal budget		$\boldsymbol{E = 100{,}000}$

Section 3.7 of the methodology.

4.6 B2 margin threshold calibration

The Branch 2 veto gate fires when the world model's preferred candidate differs from the policy's preferred candidate and its score advantage exceeds a fixed margin threshold (Section 3.2). The threshold must be calibrated: a value of zero causes the world model to veto the policy whenever it disagrees regardless of confidence margin, while a very high value prevents any veto and collapses B2 to B1.

A sweep over candidate threshold values was conducted on the validation split of `train_part1_2` before any test-set evaluation. The swept values were $\{0.05, 0.10, 0.25, 0.50\}$ for the V-JEPA2-AC guard and $\{0.25, 0.50, 1.00\}$ for the LeWM guard. For each candidate value, the guard was run over the validation windows, and the nonlinear acceptance rate (the fraction of windows where the veto fires) together with the validation win rate against B1 were recorded. The selected threshold is the value that maximises the validation win rate subject to a nonlinear acceptance rate that remains below 50 per cent of windows, ensuring the world model intervenes selectively rather than on the majority of decisions.

The threshold calibration uses only the validation split. The test split is not consulted at any point during threshold selection, and the threshold is frozen before the final test-set evaluation begins. This ensures that the B2 result on the test split is not contaminated by information derived from test-set distributions.

4.7 Guard evaluation

4.7.1 Branch 1: policy-only baseline

Branch 1 records the candidate that the policy scores highest by its own internal preference for each test window. For ACT, this is the candidate with the lowest CVAE reconstruction loss among the five sampled chunks; for Diffusion Policy, it is the candidate with the lowest denoising residual among the five independent chains. No world model is queried.

4.7.2 Branch 2: V-JEPA2-AC veto guard

The Branch 2 guard scores each of the five candidate action chunks per window as follows. The frozen V-JEPA2 encoder processes the observation frame sequence and produces a current-state embedding. The action-conditioned transition head predicts the next embedding for each candidate by taking the current embedding together with the candidate's bimanual wrist state delta, normalised using the training-split statistics.

The predicted next embedding is compared against the support distribution of stored training embeddings, and a combined score is assembled from the latent prediction quality and an action out-of-distribution penalty. If the world model's highest-scoring candidate differs from the policy's highest-scoring candidate and its score advantage exceeds the calibrated margin threshold, the world model overrides the selection; otherwise the policy's preferred candidate is retained.

The same guard structure is applied independently to each world model family in the benchmark. DexWM uses its CDiT predictor operating on DINOv2 patch tokens rather than a single global embedding. LeWM uses its ViT-Tiny encoder and autoregressive predictor; the guard must first encode a rollout prefix through the predictor to establish an appropriate latent context, which requires reading the pre-exported HDF5 episode sequences. The V-JEPA2 no-AC and PointWorld guards use their respective proxy scoring functions described in Sections 4.4.4 and 4.4.5.

4.7.3 Branch 3: world-model argmax

Branch 3 operates over the frozen candidate pool produced during the Branch 2 pass for each window. The policy is not re-run; instead, the world model scores all five candidates from the B2 pool without a margin threshold and selects the candidate with the highest score unconditionally. The correctness of this design can be verified through the guard manifest: every B3 run should record all 7,607 test windows as decided by the world model (override count equals total window count), with zero windows falling back to the policy preference. This manifest invariant was used to detect and correct the Branch 3 selection-mode error documented in Section 4.9.

4.7.4 Manifest and checkpoint design

Every guard run emits a frozen manifest at completion recording the selection mode, candidate source, aggregate score statistics, and per-reason override counts. Guard runs also write per-window JSONL checkpoints flushed after each window, allowing interrupted runs of thousands of windows to be resumed from the last completed window via a `--resume-dir` flag rather than restarted from scratch.

4.8 Evaluation safeguards

Oracle goal elimination. In early experiments, the latent goal used for scoring was derived from the ground-truth future frame of each episode, giving Branches 2 and 3 access to information that would not be available at deployment time and artificially inflating their scores relative to Branch 1. This was corrected by setting `goal_source=none` in all canonical benchmark runs, removing the goal latent from the scoring function entirely. All results in the results chapter use this corrected configuration.

Candidate pool invariant. All branches score exactly the same set of five candidates per window. Branch 3 uses a frozen manifest of the candidate sets generated during the Branch 2 pass to guarantee that the same five candidates are evaluated under both selection modes. This prevents the Branch 2 versus Branch 3 comparison from being confounded by differences in candidate generation.

Shared comparison surface. The `policy_chunk_compare` stage computes all cross-branch and cross-family metrics on the same prediction targets rather than relying on controller-native summaries, which measure different quantities for different families. This ensures that the translation L2 and rotation error values reported in the results chapter are computed identically across all configurations.

Test-split isolation. The test split was held out throughout all stages of the project. All training, hyperparameter selection, threshold calibration, and qualitative development work used training and validation splits only. The test split was not consulted during any design decision.

4.9 Implementation challenges and resolutions

Several significant implementation challenges arose during the project and were diagnosed and resolved before the final benchmark results were produced.

V-JEPA2 extraction window cap. The initial Phase 1 extraction applied a default cap of 2,000 windows per split. For a split with 140,000 or more windows this produced approximately 1,000 training pairs after the 90/10 train/validation split, representing roughly a 95-fold shortfall relative to the intended 94,000 pairs. All V-JEPA2-AC transition head runs conducted with the capped extraction were invalid and were re-run after the `--full` flag was added to the extraction entry point to disable the cap. The re-runs produced results consistent with the convergence study predictions.

LeWM CPU inference. An early LeWM guard run placed the model on CPU despite the container having GPU access, because the model-loading code contained a hard-coded `.to("cpu")` call. Running 7,607 windows with five candidates each and multiple rollout prefix steps entirely on CPU was estimated to require approximately 54 hours per guard run. The fix checked CUDA availability at load time and moved the model and all tensors to the GPU accordingly, achieving approximately 19.5 windows per second and reducing the estimated guard run time to under seven minutes.

LeWM episode coverage defect. The initial LeWM export used `max_frames_per_episode=256`. When the benchmark aligner attempted to reconstruct windows whose start positions lay beyond frame 256 of any episode, no matching data existed in the export and those windows were silently excluded. The result was that the first LeWM evaluation covered only 160 of 7,607 intended test windows (2.1 per cent coverage). The fix computed the minimum safe frame cap from the actual benchmark window lists: 5,152 frames for the test split and 5,280 frames for `train_part1_2`. A contract of `max_frames_per_episode=5280` was adopted and the export and training were rebuilt, raising coverage to 100 per cent.

LeWM CPU-only training shortfall. A separate early LeWM training run omitted the `--train-accelerator gpu` and `--train-devices 4` flags, causing training to run on a single CPU. Under the effective items formula $E = S \times B \times G \times A$, a run on one device with the same step count as a four-device run accumulates four times fewer effective items than intended. All LeWM training results produced under this configuration were invalidated and re-run with the correct flags, producing results consistent with the convergence study.

DexWM guard dimension mismatch. An early DexWM guard run loaded DINOv2 ViT-Base (producing 768-dimensional patch tokens) rather than the architecturally required ViT-Large (1,024-dimensional tokens). The mismatch caused all guard inference to fail with a dimension error; because the codebase contained a silent fallback producing zero scores, this failure was not immediately visible in the output. The fix removed the silent fallback (replacing it with an explicit hard-failure) and confirmed the correct ViT-Large configuration throughout.

Branch 3 selection mode error. An early Branch 3 guard run launched without the `--selection-mode world_model_only` flag. The default mode is `best_score`, which combines policy and world model scoring; under this mode with the same candidate pool as Branch 2, Branch 3 produced outputs identical to Branch 2 in all 7,607 windows. The error was detected through the pairwise comparison output showing a 0.000 win rate for Branch 3 against Branch 2, indicating identity rather than competition. The fix re-ran Branch 3 with the correct flag, verified by inspecting the manifest override count: 7,607 windows decided by the world model against 0 policy fallbacks.

Oracle goal leak. The goal latent leak described in Section 4.8 caused inflated B2 and B3 scores in early experiments. Detection required comparing per-window score breakdowns and noticing that the goal-distance term was being computed against the ground-truth future embedding. Correction required re-running all canonical benchmark guard stages with `goal_source=none`.

HDF5 file handle exhaustion. The LeWM guard initially opened the exported `episodes.h5` file inside the per-window scoring function. With 7,607 windows per test-set guard run, this produced approximately 38,000 file open and close operations, risking descriptor exhaustion on systems with low limits. The fix opened the HDF5 file once during runtime initialisation and stored the handle for the duration of the run, reducing file operations to one.

4.10 Auxiliary modules

Two auxiliary modules are fully implemented in the repository but were excluded from the final benchmark evaluation.

The semantic alignment stage converts indexed windows into captions, structured tags, and text embeddings. The pipeline uses a Vision Language Model (VLM) to generate free-form descriptions and task-relevant categorical tags for each window, followed by a separate embedding model that maps the text to a shared representation space. A critic-and-retry loop rejects low-quality outputs and requests regenerated captions.

The object grounding and mask generation stage consumes semantic object prompts from the alignment stage and produces bounding box detections and segmentation masks for identified objects. The current implementation provides pixel-space localisation of objects but does not estimate SE(3) object pose.

Both modules were excluded from the primary benchmark because they could only be wired into one of the two world model families without the other having access to equivalent inputs, violating the shared input contract established in Section 3.5.2. Their implementation is documented in Appendix **??**.

References

[1] B. D. Argall, S. Chernova, M. Veloso, and B. Browning, "A survey of robot learning from demonstration," *Robotics and Autonomous Systems*, vol. 57, no. 5, pp. 469–483, 2009. DOI: 10.1016/j.robot.2008.10.024

[2] M. Jeannerod, "The timing of natural prehension movements," *Journal of Motor Behavior*, vol. 16, no. 3, pp. 235–254, 1984. DOI: 10.1080/00222895.1984.10735319

[3] M. Jeannerod, M. A. Arbib, G. Rizzolatti, and H. Sakata, "Grasping objects: The cortical mechanisms of visuomotor transformation," *Trends in Neurosciences*, vol. 18, no. 7, pp. 314–320, 1995. DOI: 10.1016/0166-2236(95)93921-J

[4] M. Santello, M. Flanders, and J. F. Soechting, "Postural hand synergies for tool use," *Journal of Neuroscience*, vol. 18, no. 23, pp. 10 105–10 115, 1998. DOI: 10.1523/JNEUROSCI.18-23-10105.1998

[5] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 15, 2011.

[6] Z. Ding, "Imitation learning," in *Deep Reinforcement Learning: Fundamentals, Research and Applications*, H. Dong, Z. Ding, and S. Zhang, Eds. Singapore: Springer Singapore, 2020, pp. 273–306, ISBN: 978-981-15-4095-0. DOI: 10.1007/978-981-15-4095-0_8 [Online]. Available: https://doi.org/10.1007/978-981-15-4095-0_8

[7] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, *Learning fine-grained bimanual manipulation with low-cost hardware*, Apr. 2023. DOI: 10.48550/arXiv.2304.13705 arXiv: 2304.13705 [cs].

[8] C. Chi et al., *Diffusion policy: Visuomotor policy learning via action diffusion*, Mar. 2024. DOI: 10.48550/arXiv.2303.04137 arXiv: 2303.04137 [cs].

[9] S. Jain et al., *DiWA: Diffusion policy adaptation with world models*, ArXiv preprint, Aug. 2025. arXiv: 2508.03645 [cs].

[10] K. M. Lynch and F. C. Park, *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press, 2017, ISBN: 978-1-107-15630-2.

[11] Omitram, *Left-hand vs. right-hand coordinate systems: The 3d developer's guide - omitram — omitram.com*, https://omitram.com/3d-coordinate-systems-left-vs-right-hand/, 2024.

[12] Y. Zhou, C. Barnes, J. Lu, J. Yang, and H. Li, "On the continuity of rotation representations in neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 5745–5753. DOI: `10.1109/CVPR.2019.00589`

[13] N. Grossmann, E. Gröller, and M. Waldner, "Concept splatters: Exploration of latent spaces based on human interpretable concepts," *Computers & Graphics*, vol. 105, pp. 73–84, 2022, ISSN: 0097-8493. DOI: `https://doi.org/10.1016/j.cag.2022.04.013` [Online]. Available: `https://www.sciencedirect.com/science/article/pii/S0097849322000656`

[14] A. Dawid and Y. LeCun, "Introduction to latent variable energy-based models: A path towards autonomous machine intelligence," *Journal of Statistical Mechanics: Theory and Experiment*, 2023. arXiv: `2306.02572 [cs]`.

[15] D. P. Kingma and M. Welling, *Auto-encoding variational bayes*, 2013. DOI: `10.48550/arXiv.1312.6114` arXiv: `1312.6114 [stat.ML]`.

[16] L. Li and R. Betti, "A machine learning-based data augmentation strategy for structural damage classification in civil infrastructure system," *Journal of Civil Structural Health Monitoring*, vol. 13, pp. 1–21, May 2023. DOI: `10.1007/s13349-023-00705-5`

[17] AnotherDatum. "Variational autoencoders explained," Accessed: Apr. 24, 2026. [Online]. Available: `https://anotherdatum.com/vae.html`

[18] K. Sohn, H. Lee, and X. Yan, "Learning structured output representation using deep conditional generative models," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[19] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. DOI: `10.48550/arXiv.1706.03762` arXiv: `1706.03762 [cs.CL]`.

[20] S. Panda, *Decoding attention types and strategies for effective nlp models*, `https://www.linkedin.com/pulse/decoding-attention-types-strategies-effective-nlp-models-swagat-panda-erhef/`, 2024.

[21] M. Assran et al., "Self-supervised learning from images with a joint-embedding predictive architecture," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023. arXiv: `2301.08243 [cs]`.

[22] IBM Developer, *Models for machine learning*, `https://developer.ibm.com/articles/cc-models-machine-learning/`, 2017. Accessed: May 14, 2026.

[23] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, *Film: Visual reasoning with a general conditioning layer*, 2017. arXiv: 1709.07871 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1709.07871

[24] *MTDP: Modulated Transformer Diffusion Policy Model*, https://arxiv.org/html/2502.09029v1. Accessed: Apr. 26, 2026.

[25] M. Reuss, J. Pari, P. Agrawal, and R. Lioutikov, *Efficient diffusion transformer policies with mixture of expert denoisers for multitask learning*, 2024. arXiv: 2412.12953 [cs.LG]. [Online]. Available: https://arxiv.org/abs/2412.12953

[26] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, *Mastering diverse domains through world models*, Nature, 2025, 2023. arXiv: 2301.04104 [cs].

[27] J. Quevedo, A. K. Sharma, Y. Sun, V. Suryavanshi, P. Liang, and S. Yang, *WorldGym: World model as an environment for policy evaluation*, Sep. 2025. DOI: 10.48550/arXiv.2506.00613 arXiv: 2506.00613 [cs].

[28] Q. Bu et al., *Closed-loop visuomotor control with generative expectation for robotic manipulation*, Oct. 2024. DOI: 10.48550/arXiv.2409.09016 arXiv: 2409.09016 [cs].

[29] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta, "R3M: A universal visual representation for robot manipulation," in *Conference on Robot Learning (CoRL)*, 2022. arXiv: 2203.12601 [cs].

[30] A. Brohan et al., *RT-1: Robotics transformer for real-world control at scale*, 2022. arXiv: 2212.06817 [cs.RO]. [Online]. Available: https://arxiv.org/abs/2212.06817

[31] M. Assran et al., *V-JEPA 2: Self-supervised video models enable understanding, prediction and planning*, Jun. 2025. DOI: 10.48550/arXiv.2506.09985 arXiv: 2506.09985 [cs].

[32] N. Tsagkas, A. Sochopoulos, D. Danier, C. X. Lu, and O. Mac Aodha, *When pre-trained visual representations fall short: Limitations in visuo-motor robot learning*, 2025. arXiv: 2502.03270 [cs].

[33] R. G. Goswami et al., *World models can leverage human videos for dexterous manipulation*, Dec. 2025. DOI: 10.48550/arXiv.2512.13644 arXiv: 2512.13644 [cs].

[34] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero, *Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels*, Verified from local PDF metadata and first page, Mar. 2026. arXiv: 2603.19312 [cs.LG].

[35] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, *DINO-WM: World models on pre-trained visual features enable zero-shot planning*, 2024. arXiv: `2411.04983 [cs.LG]`. [Online]. Available: `https://arxiv.org/abs/2411.04983`

[36] W. Huang et al., *Pointworld: Scaling 3d world models for in-the-wild robotic manipulation*, 2026. arXiv: `2601.03782 [cs.RO]`. [Online]. Available: `https://arxiv.org/abs/2601.03782`

[37] N. Hansen, H. Su, and X. Wang, "TD-MPC2: Scalable, robust world models for continuous control," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv: `2310.16828 [cs]`.

[38] T. Yu et al., "MOPO: Model-based offline policy optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv: `2005.13239 [cs]`.

[39] R. Kidambi, A. Rajeswaran, P. Netrapalli, and T. Joachims, "MOReL: Model-based offline reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv: `2005.05951 [cs]`.

[40] A. Argenson and G. Dulac-Arnold, "Model-based offline planning," in *International Conference on Learning Representations (ICLR)*, 2021. arXiv: `2008.05556 [cs]`.

[41] Z. Liu, T. Swann, S. Garg, S. Levine, and A. Kumar, *Model-based offline planning with trajectory pruning*, 2021. arXiv: `2105.07351 [cs]`.

[42] X. Li et al., *Evaluating real-world robot manipulation policies in simulation*, CoRL 2024, 2024. arXiv: `2405.05941 [cs]`.

[43] Y. Li, Y. Zhu, J. Wen, C. Shen, and Y. Xu, *WorldEval: World model as real-world robot policies evaluator*, 2025. arXiv: `2505.19017 [cs]`.

[44] R. Hoque, P. Huang, D. J. Yoon, M. Sivapurapu, and J. Zhang, *EgoDex: Learning dexterous manipulation from large-scale egocentric video*, ICLR 2026, 2025. DOI: `10.48550/arXiv.2505.11709` arXiv: `2505.11709 [cs]`.

[45] University of Malta, *M.AI.ESTRO: Mixed reality AI-enhanced skill transfer and robotic optimisation*, `https://www.um.edu.mt/newspoint/news/2025/05/maiestro-mixed-reality-ai-enhanced-skill-transfer-robotic-optimisation`, University of Malta newspoint announcement, 2025.

[46] J. Song, C. Meng, and S. Ermon, *Denoising diffusion implicit models*, Oct. 2020. DOI: `10.48550/arXiv.2010.02502` arXiv: `2010.02502 [cs.LG]`.